

Convolutional Neural Networks

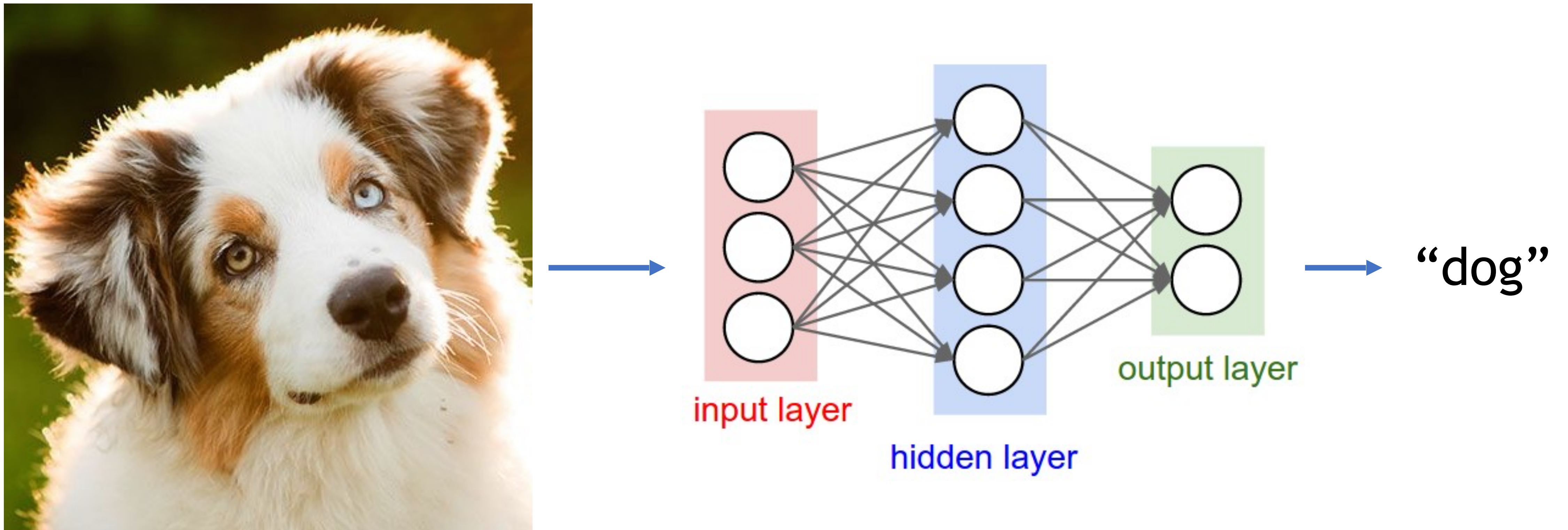
CSC 487: Deep Learning
Instructor: Jonathan Ventura

Outline

- Neural Networks with images as input
- What is a convolution?
- Pooling
- Convolution and pooling as an infinitely strong prior

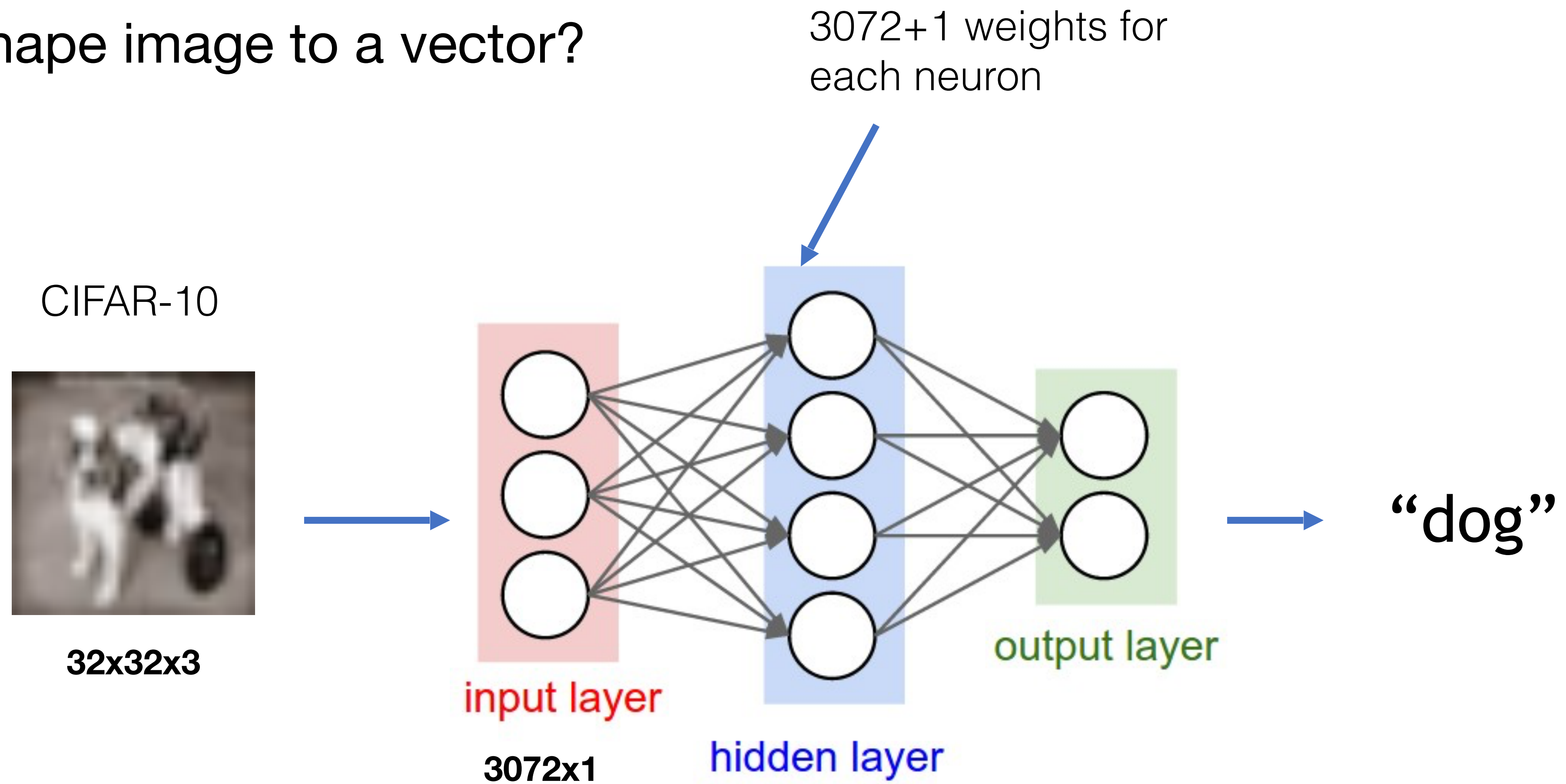
Neural Network

- How to input an image into the network?



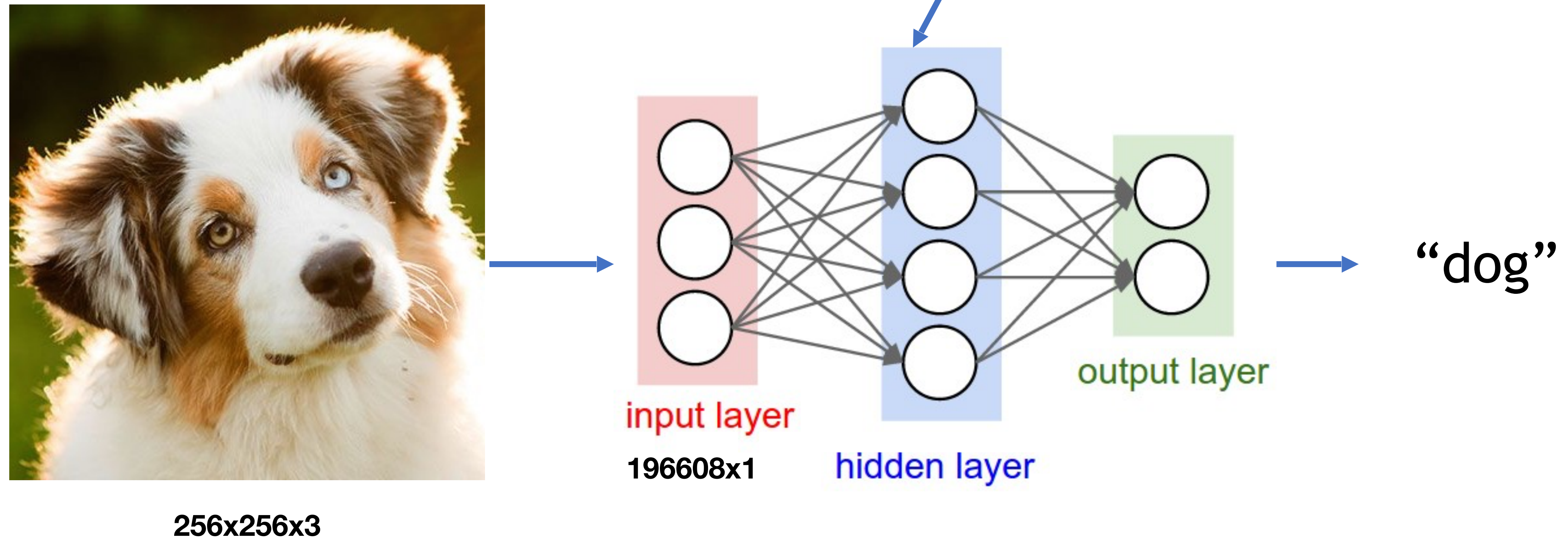
Neural Network

- Reshape image to a vector?



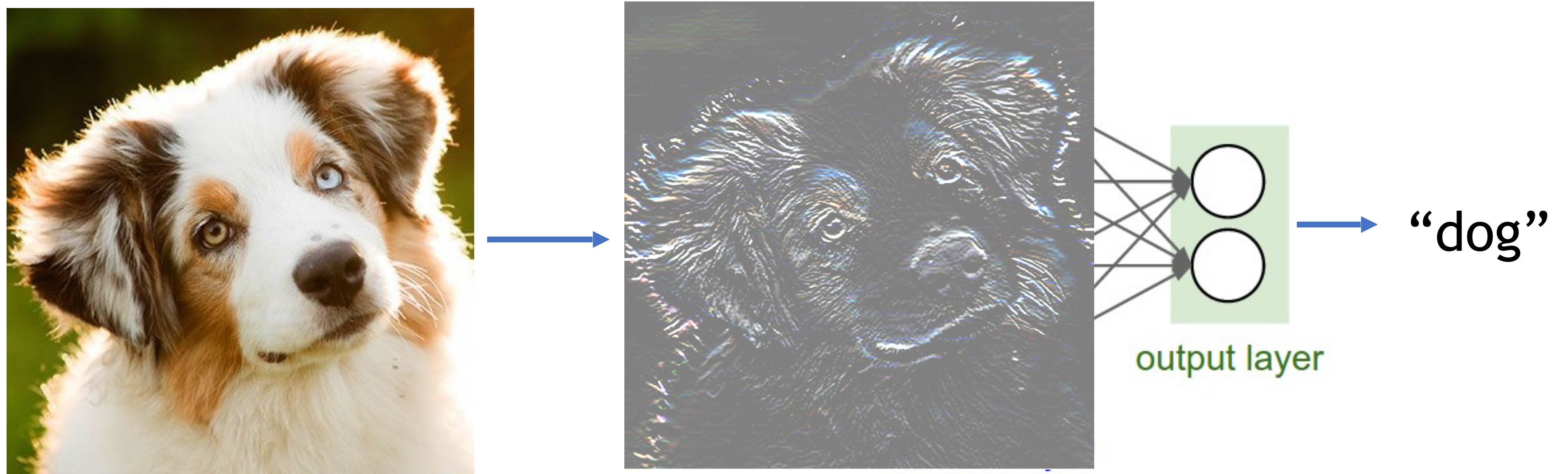
Neural Network

- Reshape image to a vector?



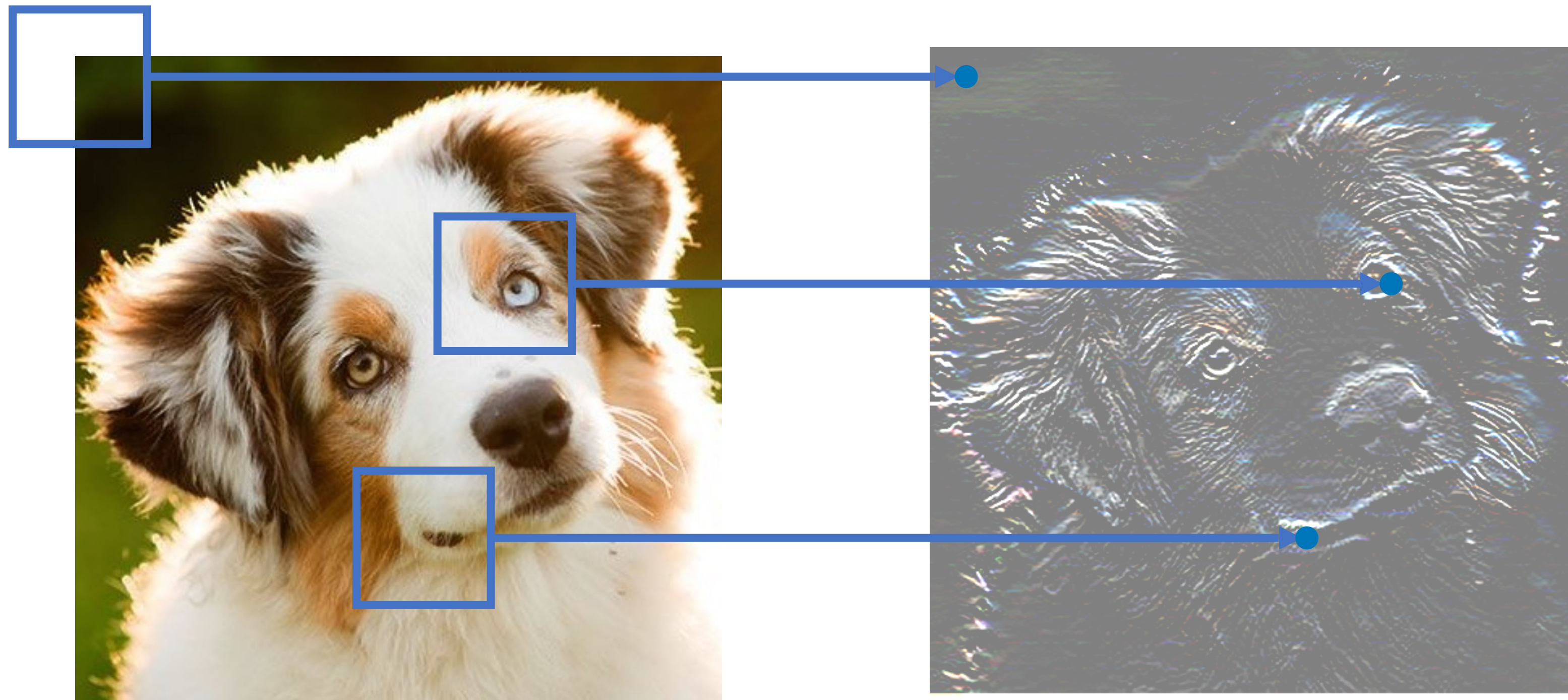
Convolutional Neural Network

- Instead of a **dot product**, basic building block is a **convolution**.



Convolution

- The convolution operation applies a filter at each location in the image



Convolution

- The filter operation is a weighted sum between the filter and a sub-window of the image

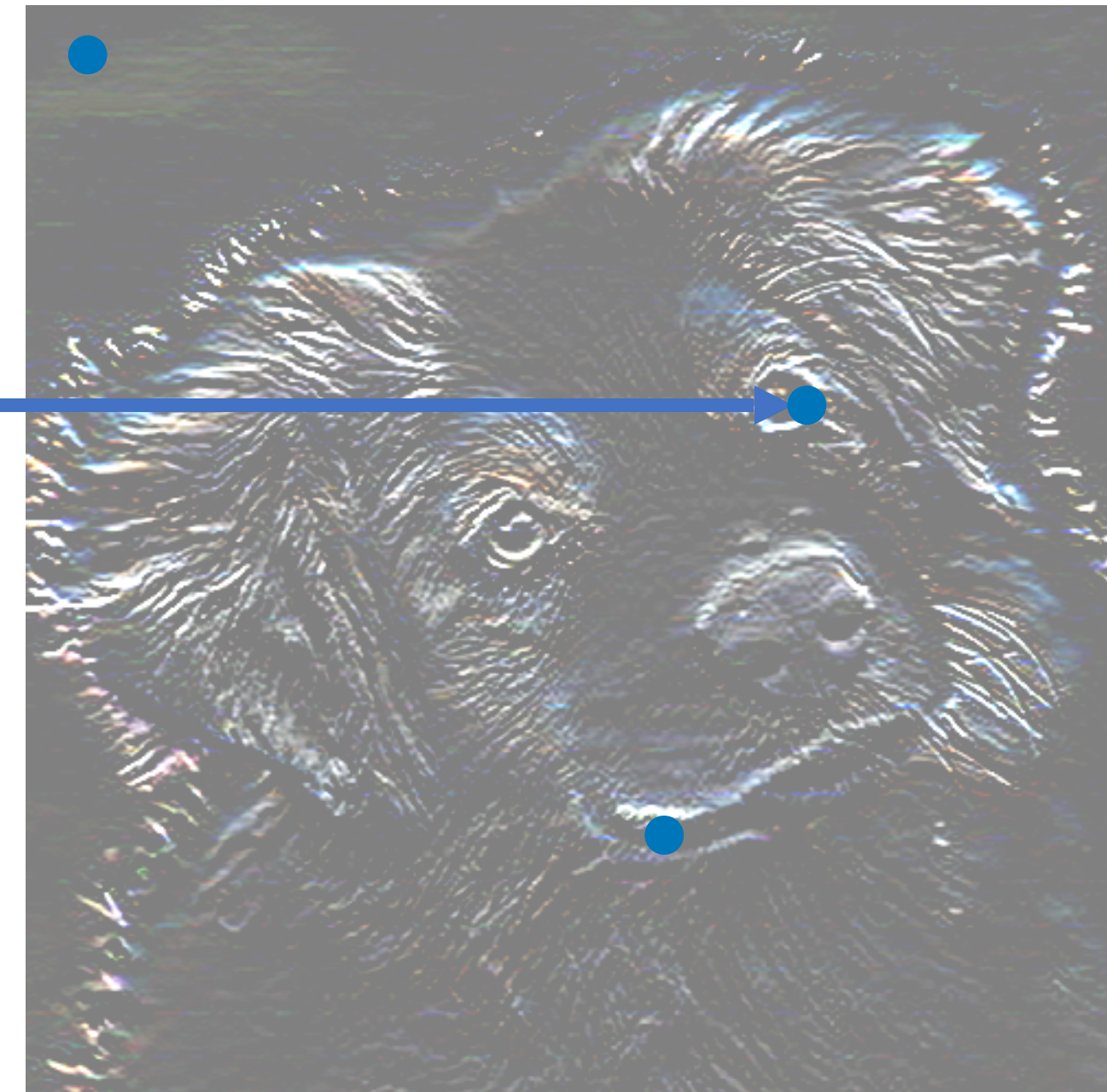
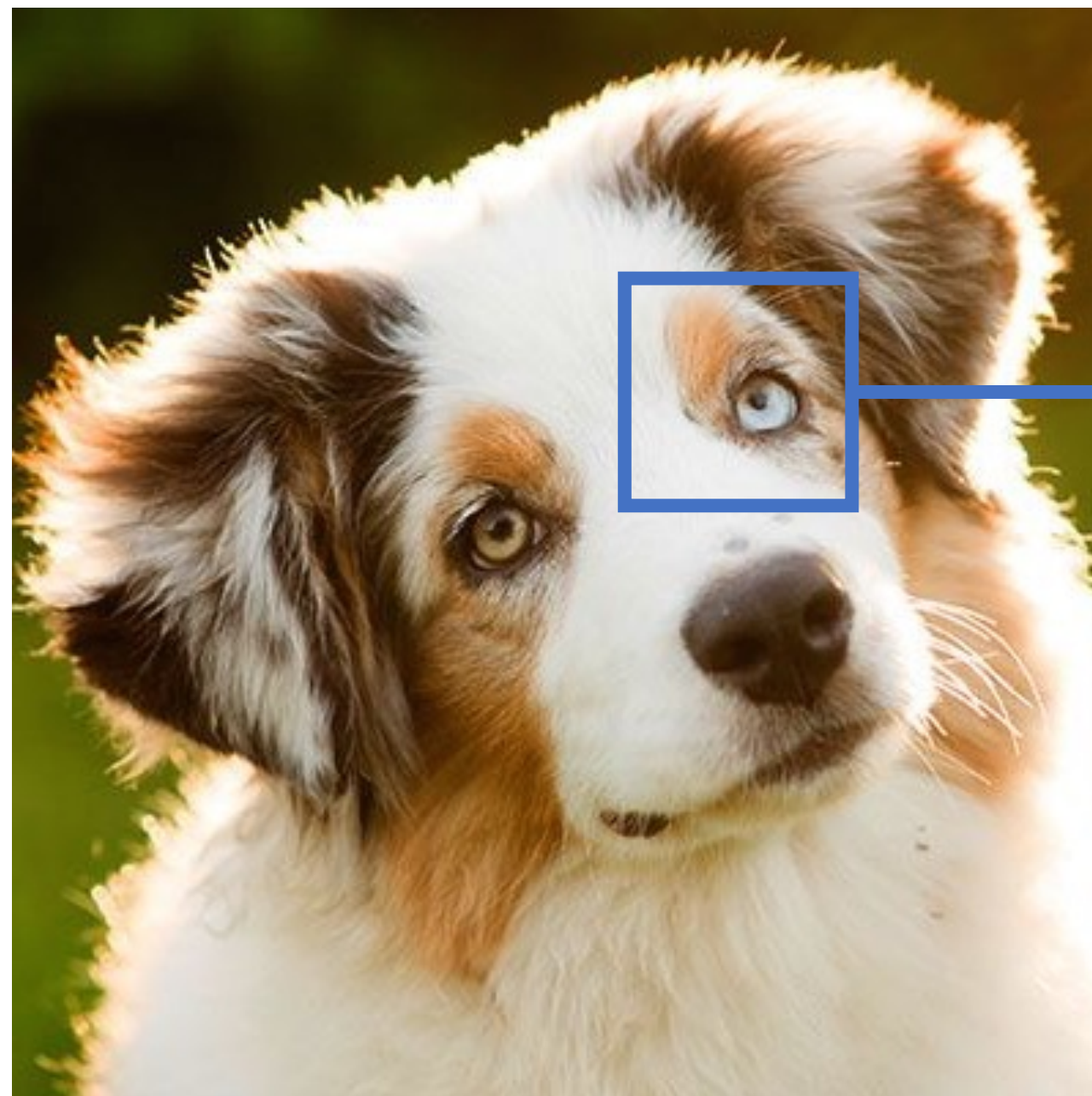
Image sub window

5	7	8
3	2	3
10	8	7

Filter matrix

-1	0	1
-1	0	1
-1	0	1

→ 0



Convolution

- The filter operation is a weighted sum between the filter and a sub-window of the image

Image sub window

5	7	8
3	2	3
10	8	7

Filter matrix

-1	0	1
-1	0	1
-1	0	1

→ 0

$$G(i, j) = \sum_{u=-k}^k \sum_{v=-k}^k H(u, v) F(i + u, j + v)$$

$$G = H * F$$

Common Convolution Operations

What do these filters do?

Filter matrix

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

Filter matrix

-1	0	1
-1	0	1
-1	0	1

Filter matrix

0	1	0
1	-4	1
0	1	0

Common Convolution Operations

What do these filters do?

Filter matrix

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

Average

Filter matrix

-1	0	1
-1	0	1
-1	0	1

dl/dx (vertical edge detector)

Filter matrix

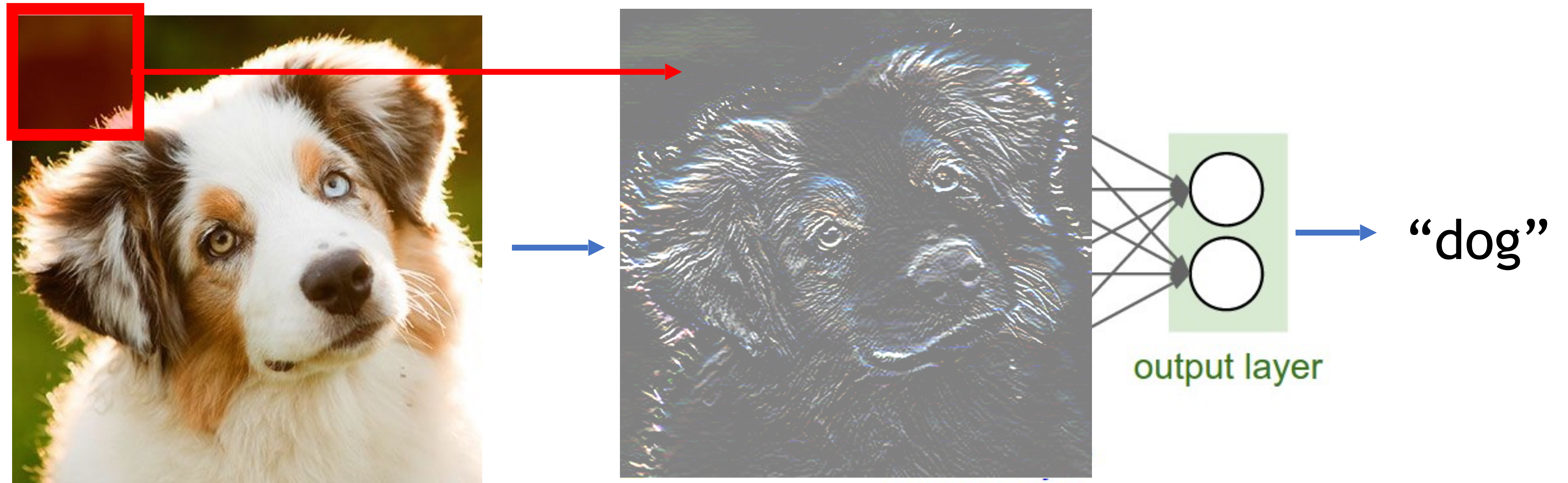
0	1	0
1	-4	1
0	1	0

Laplacian of Gaussian (high-pass)

How to make a horizontal edge detector?

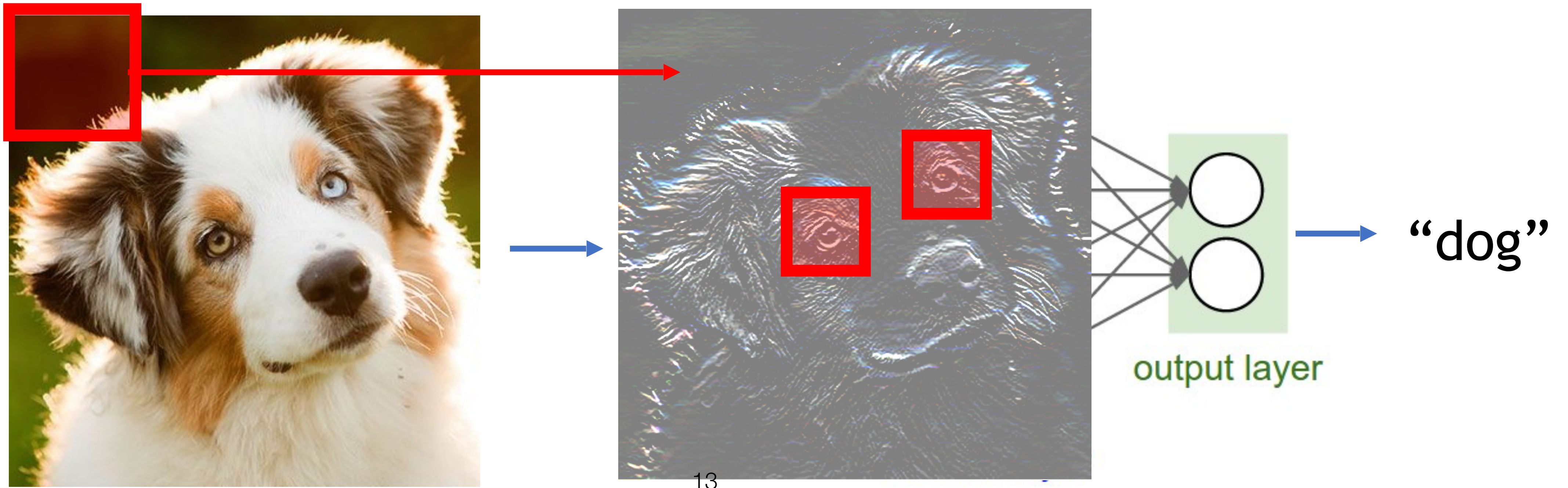
Convolutional Neural Network

- A 2D linear filter is simply a dot product applied locally at every location in the image.



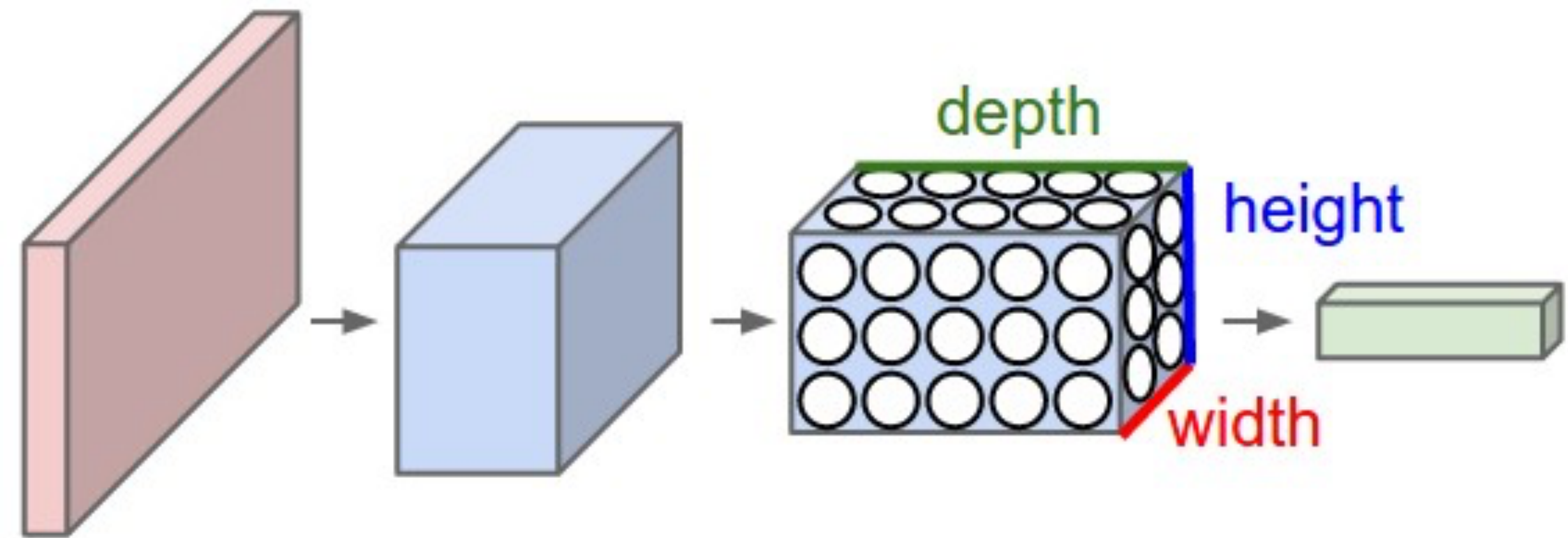
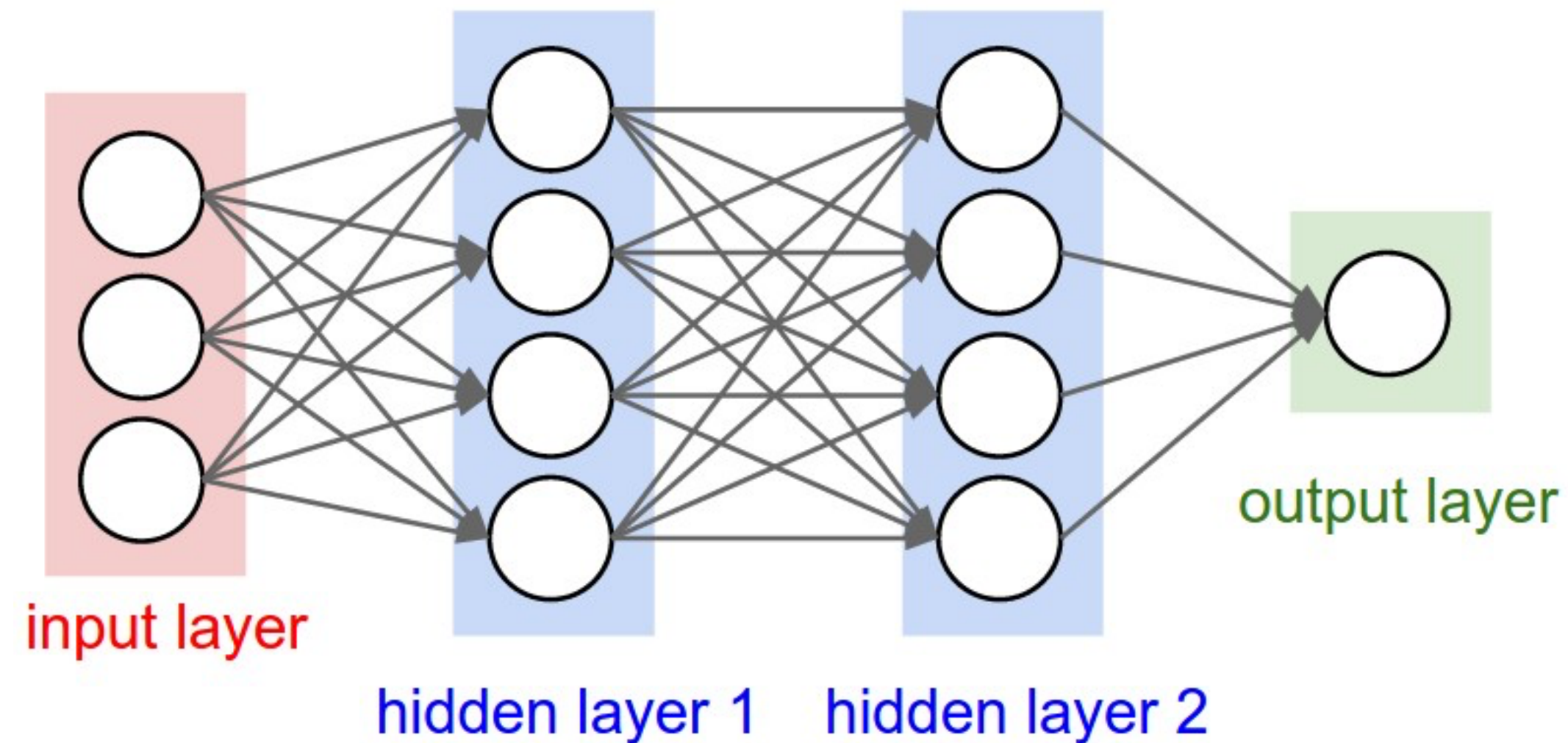
Convolutional Neural Network

- Convolution is **spatially invariant**: the same operation is applied at all locations in the image.
- This makes sense for detection & recognition, because objects and features might appear anywhere in the image.
- By combining filtering and down-sampling, the CNN can capture the local & global properties of vision problems.



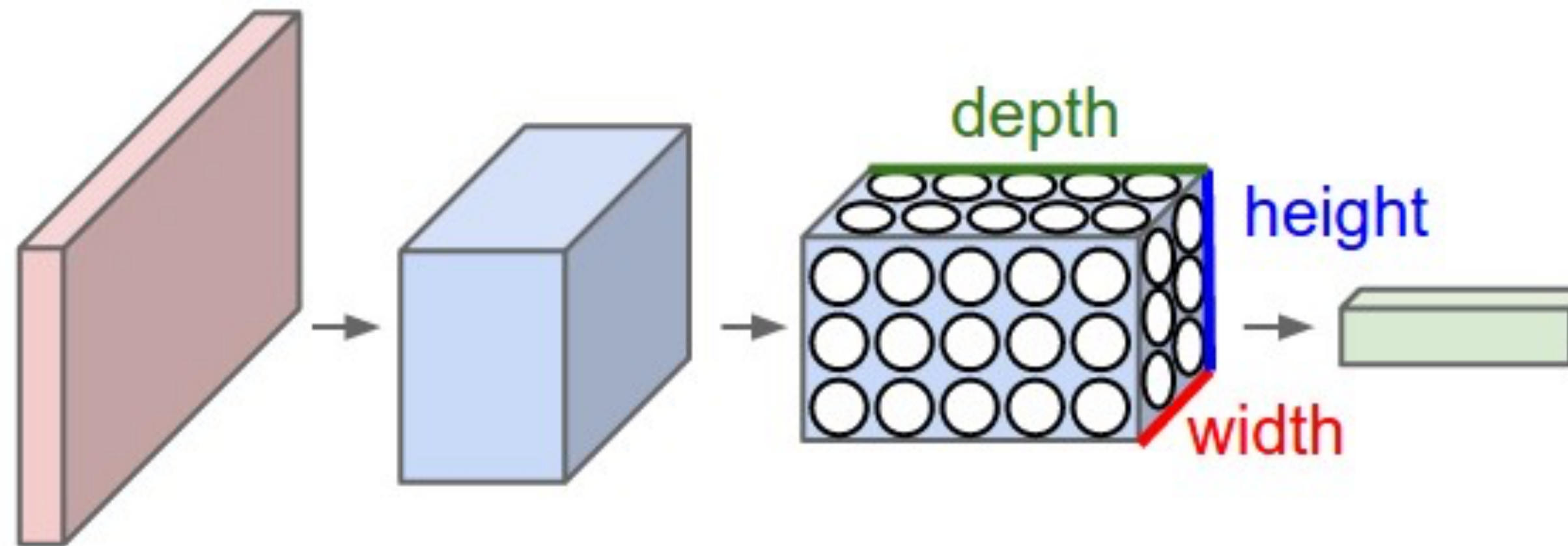
Convolutional Neural Network

- Layers are now matrices rather than vectors.
- A layer is the result of different convolutions applied to the same input.
 - Layer = applying a “bank of filters” to the input, followed by non-linear activation.



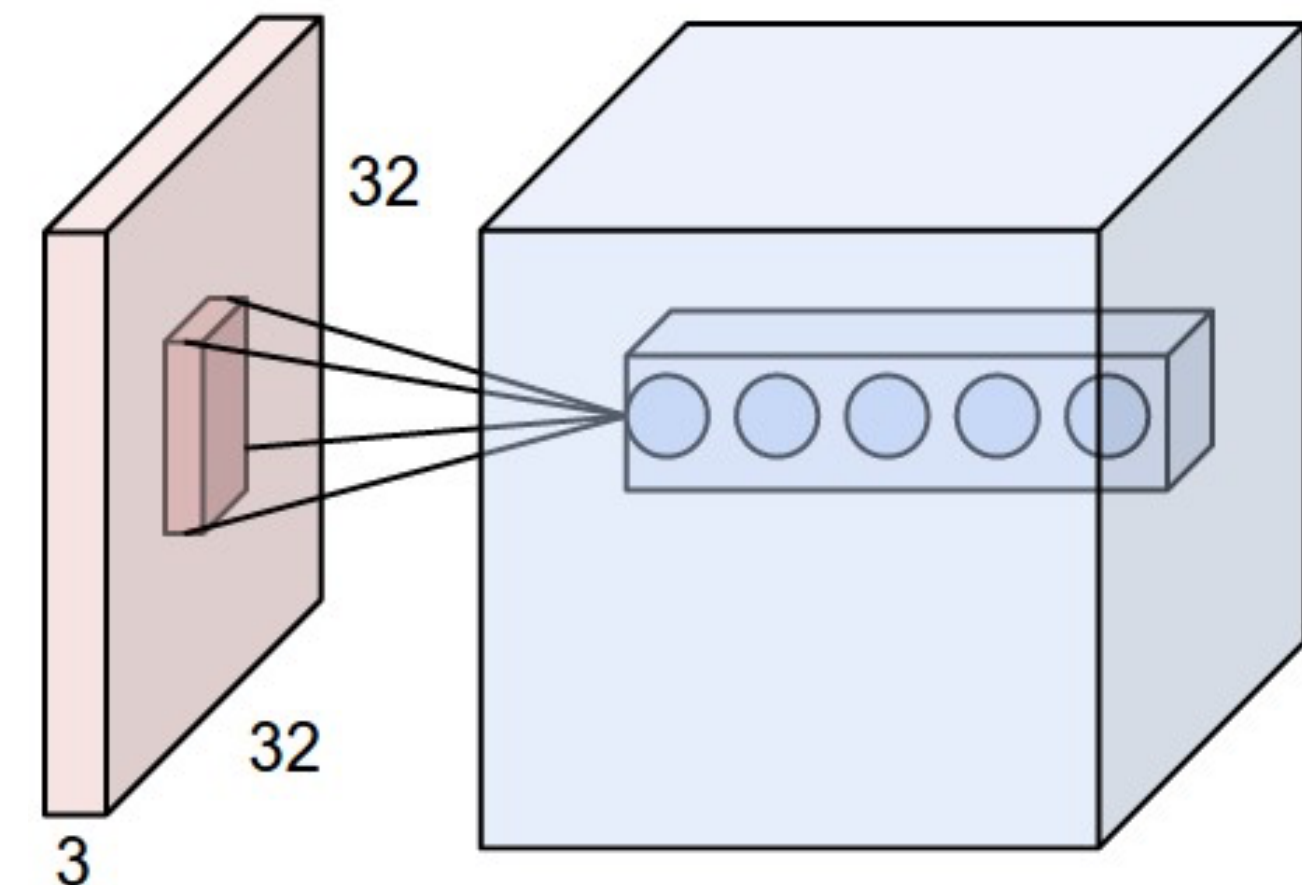
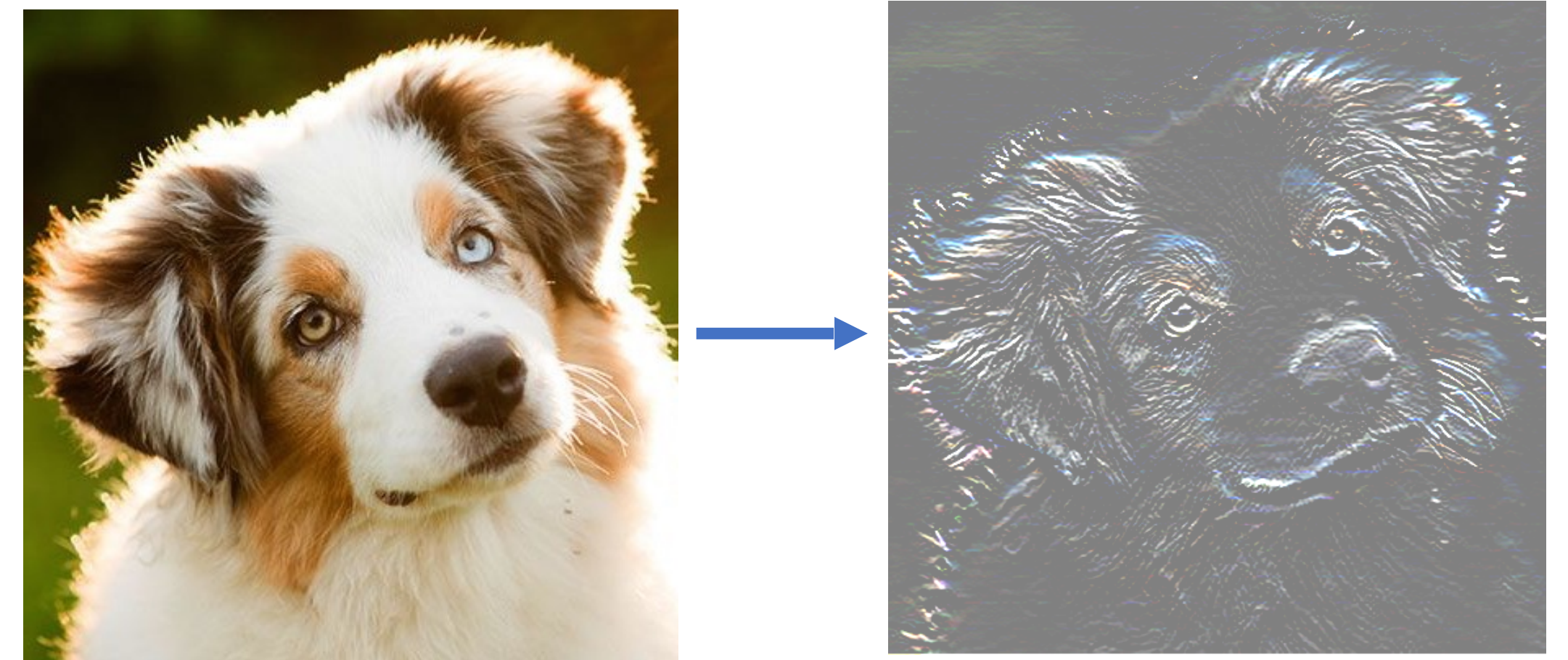
Convolutional Neural Network

- Example:
 - Input layer: 256x256 RGB image
 - Size: 256x256x3
 - Hidden layer: result of 64 3x3 convolutions
 - Size without padding: 254x254x64
 - Size with padding: 256x256x64



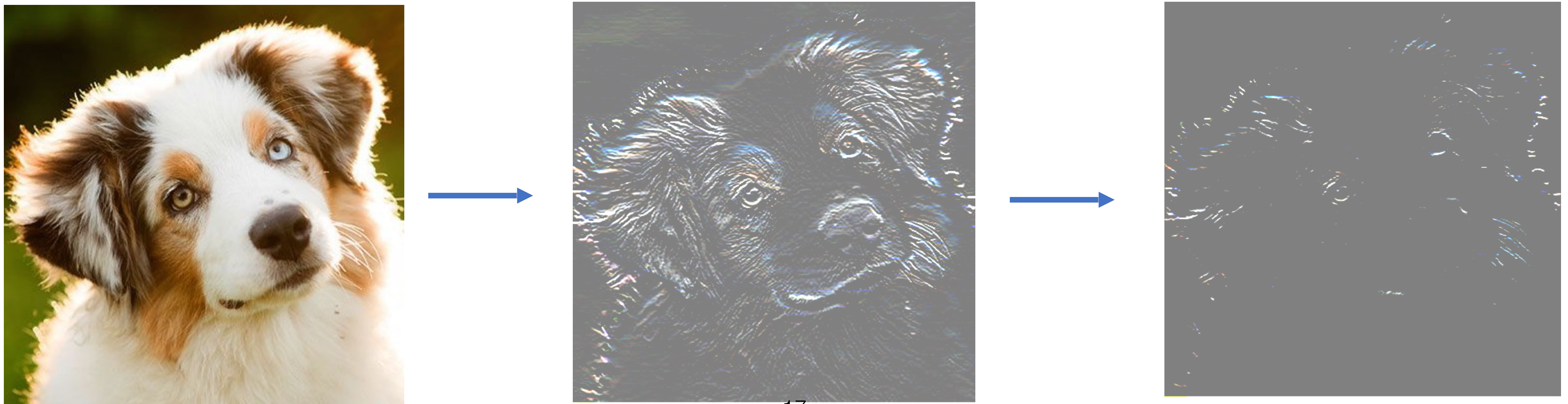
Typical CNN Layers: Convolution

- Local, spatially-invariant operation with fixed size filter (e.g. 3x3)
- Filter acts across all channels (i.e. it is actually a 3x3x3 filter)
- Can also include a bias term
- Hyperparameters:
 - Receptive field (filter size)
 - Depth (number of filters)
 - Padding (affects output size)
 - Stride (to achieve downsampling)



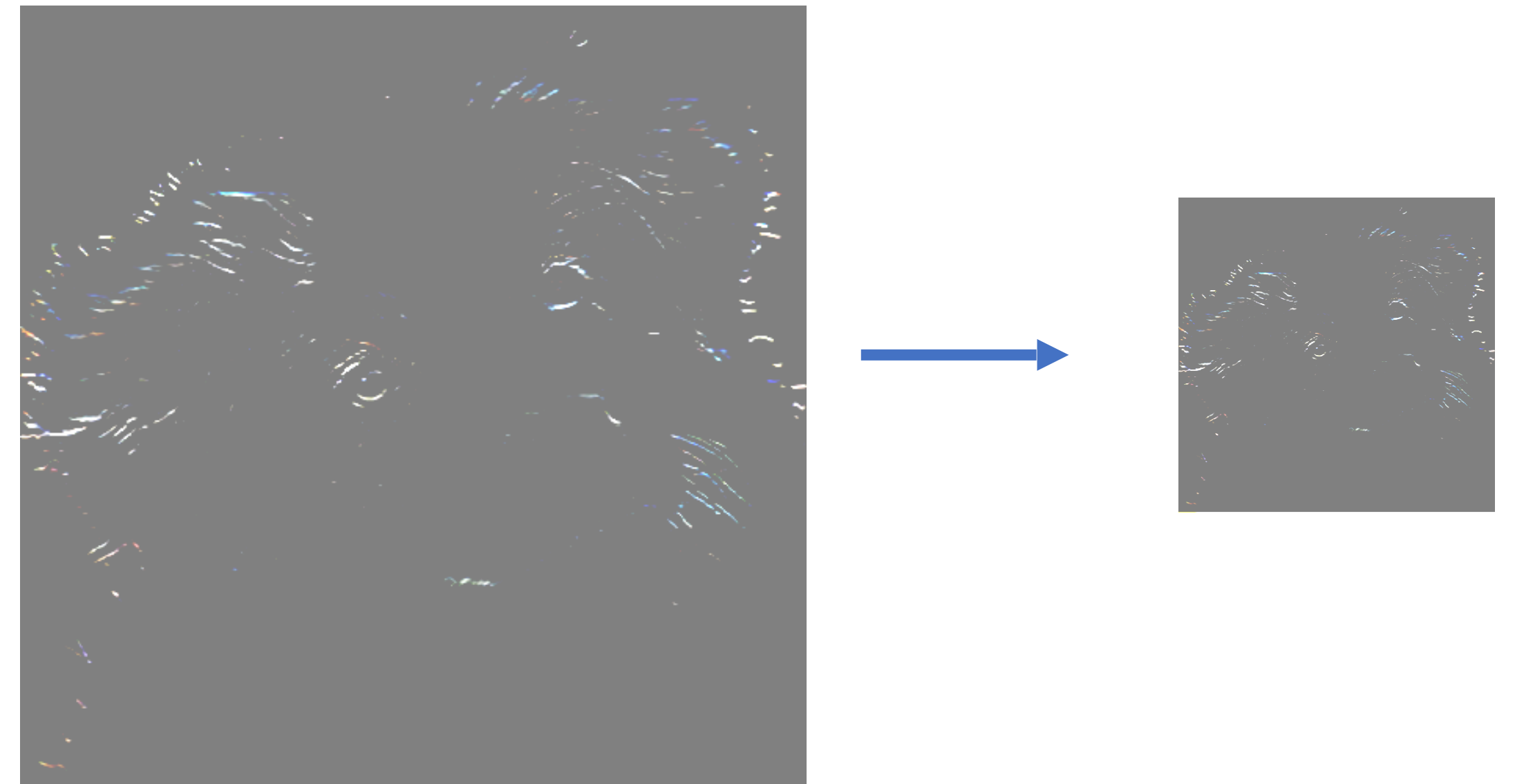
Typical CNN Layers: Non-linear Activation

- Applied to each element separately
- Examples: tanh, ReLU
- Typically no parameters or hyperparameters
- Output is same size as input

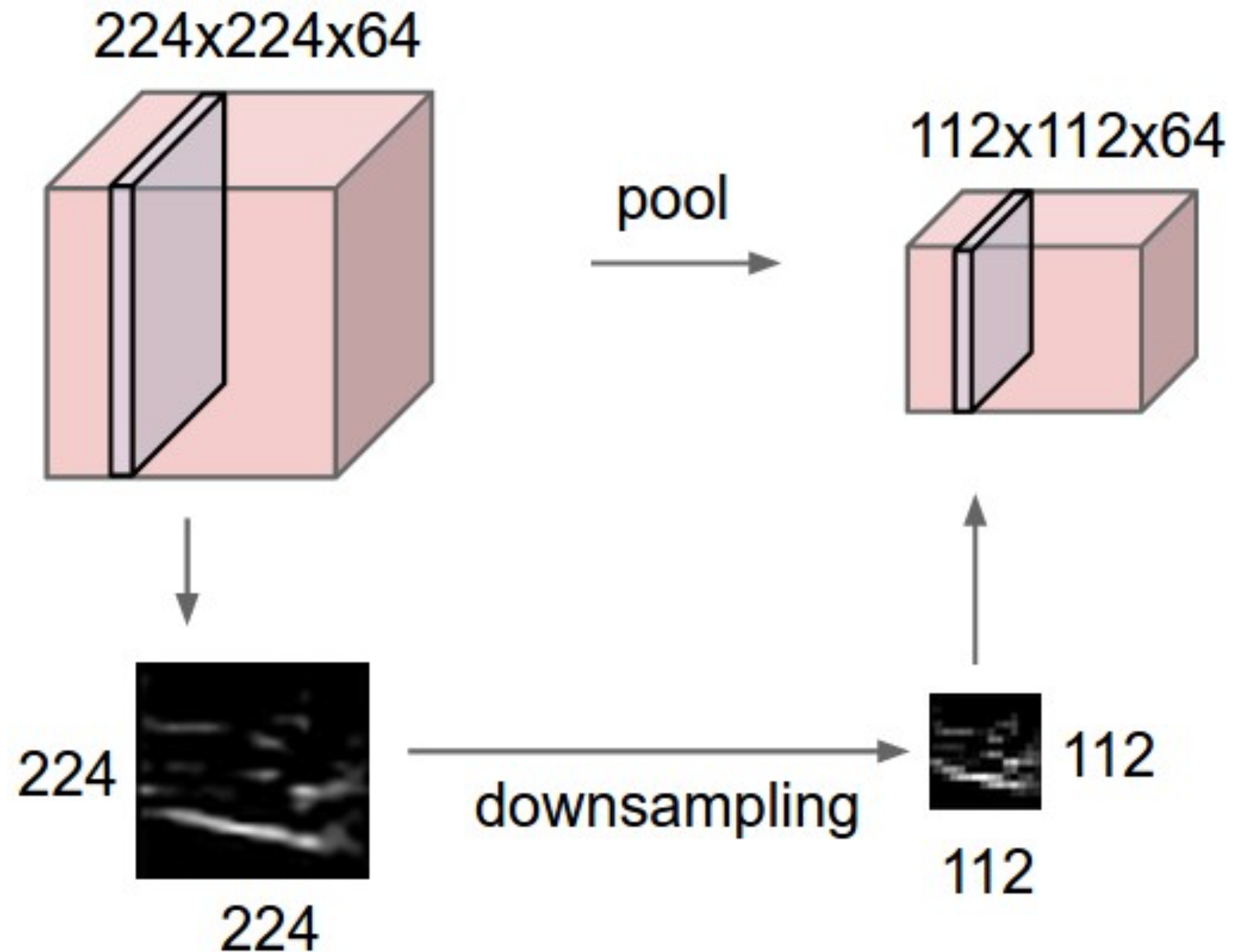


Typical CNN Layers: Pooling

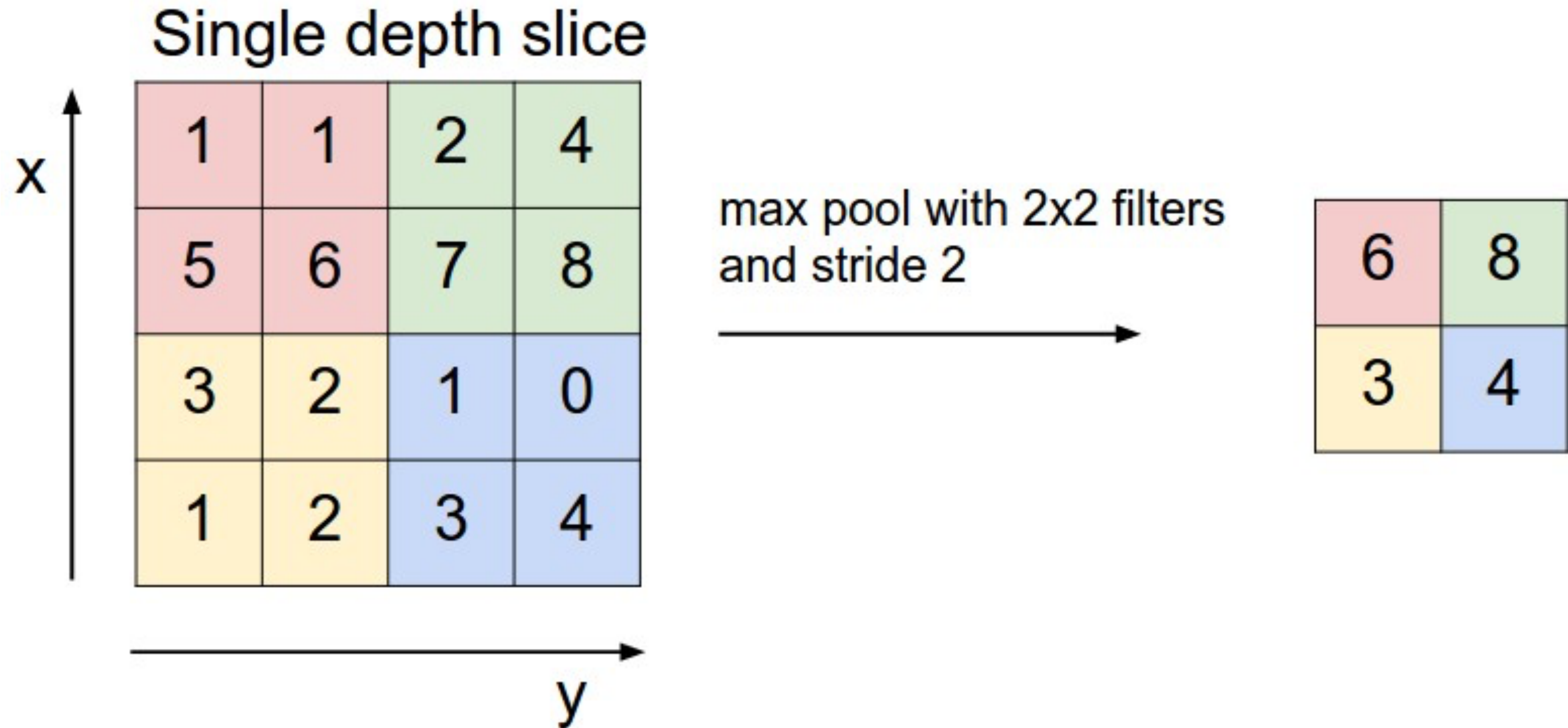
- Aggregation over a sliding window
 - Typically used for down-sampling
 - Down-samples when applied with a non-zero step size
- Examples: max pooling, average pooling, sum pooling
 - Max pooling: output maximum over a sliding window
 - Average/sum pooling: output average/sum over a sliding window
- No parameters
- Hyperparameters:
 - Receptive Field
 - Stride
- Typical settings for max pooling:
 - 2x2 with stride of 2
 - 3x3 with stride of 2 (overlapping pooling)



Max Pooling Illustration

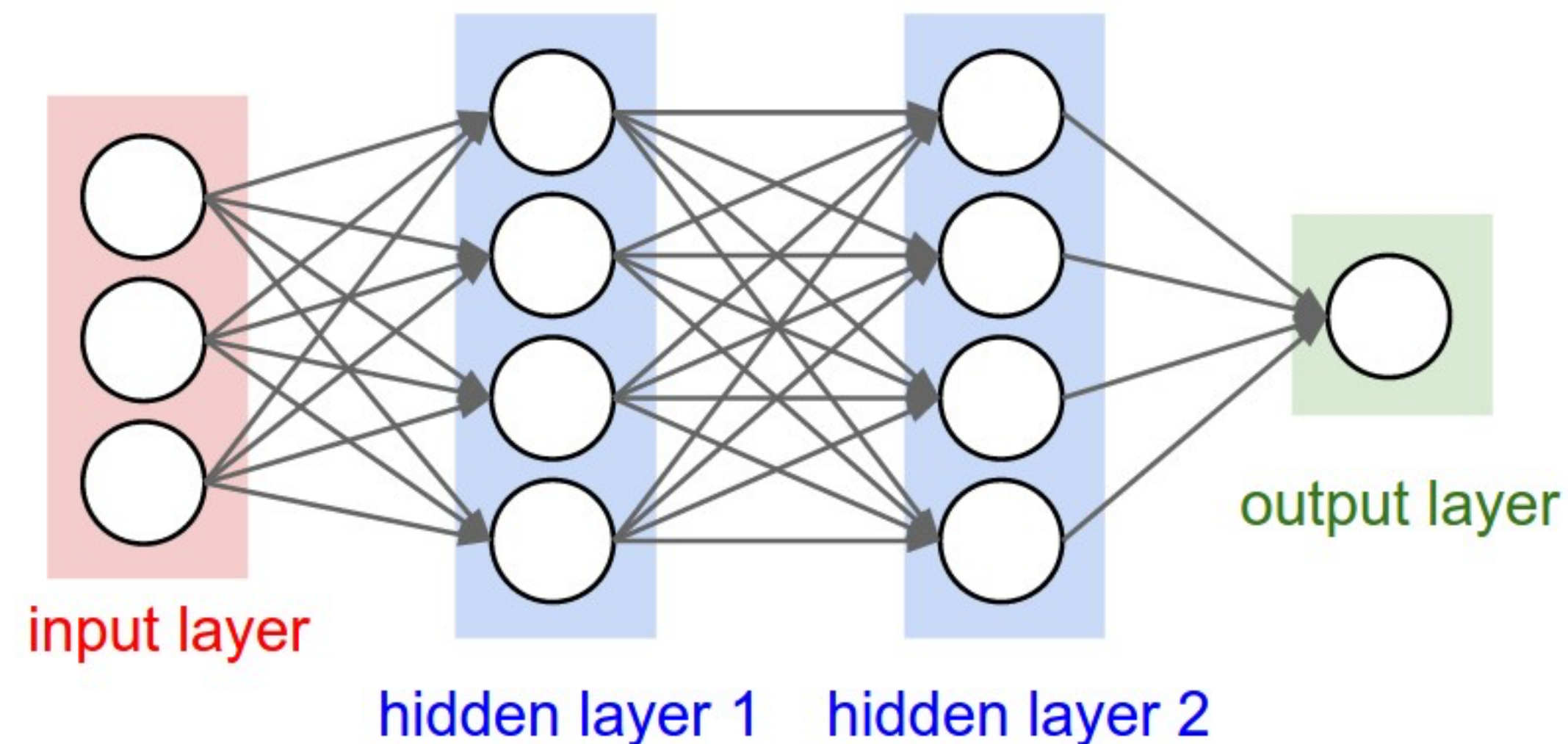


Max Pooling Illustration



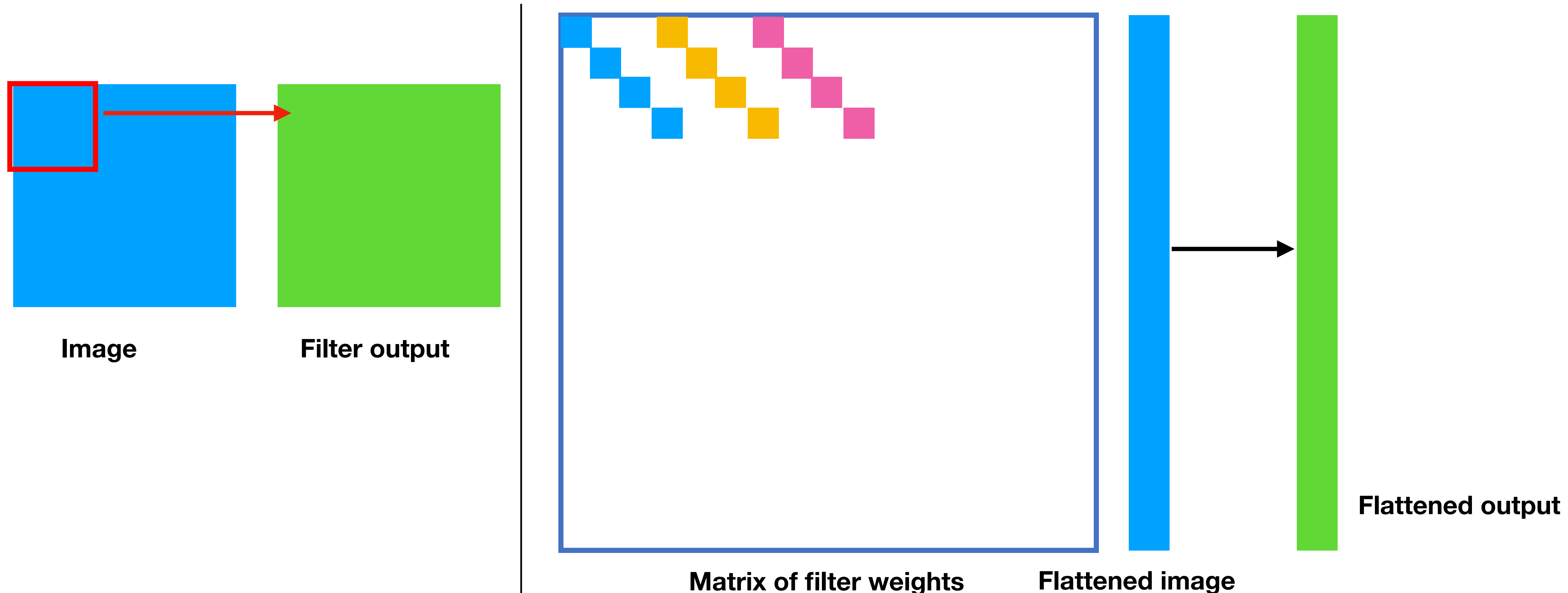
Typical CNN Layers: Fully Connected Layer

- Equivalent to an ordinary Neural Network
 - Each output neuron is “fully connected” to every input neuron
- Typically used at the end of the network
- Reduces network down to the output neurons (e.g. classes)
- Can be thought of as a convolution
 - Receptive field = entire input layer



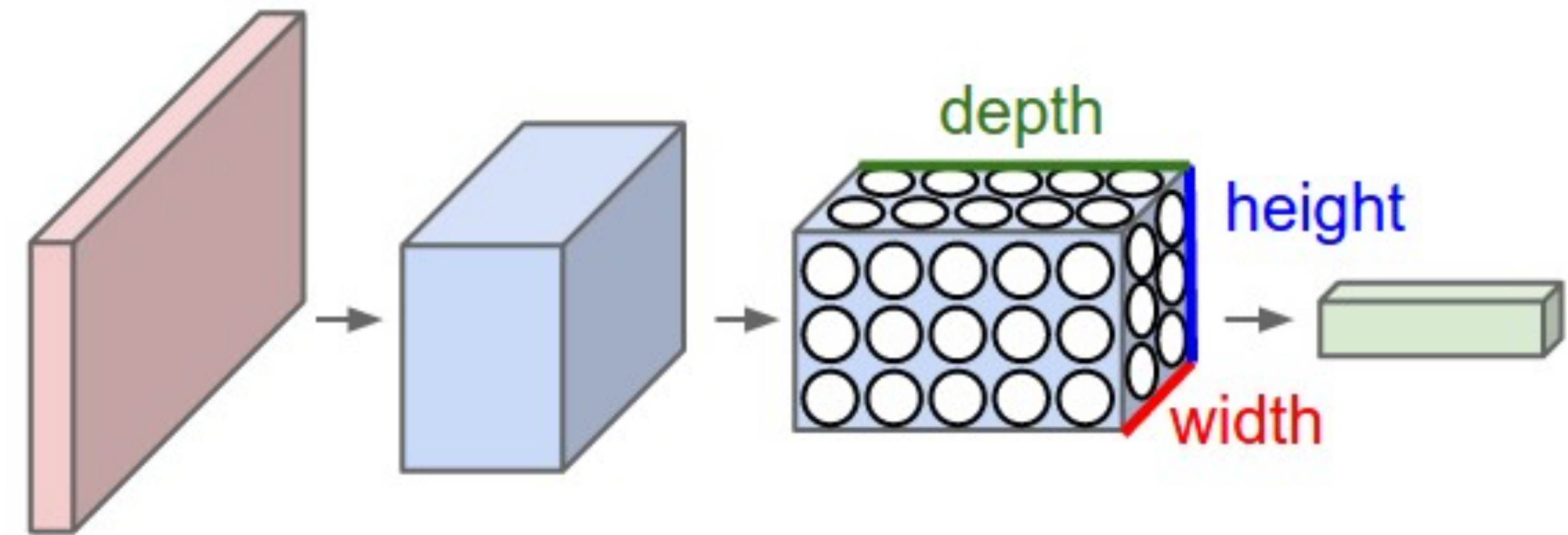
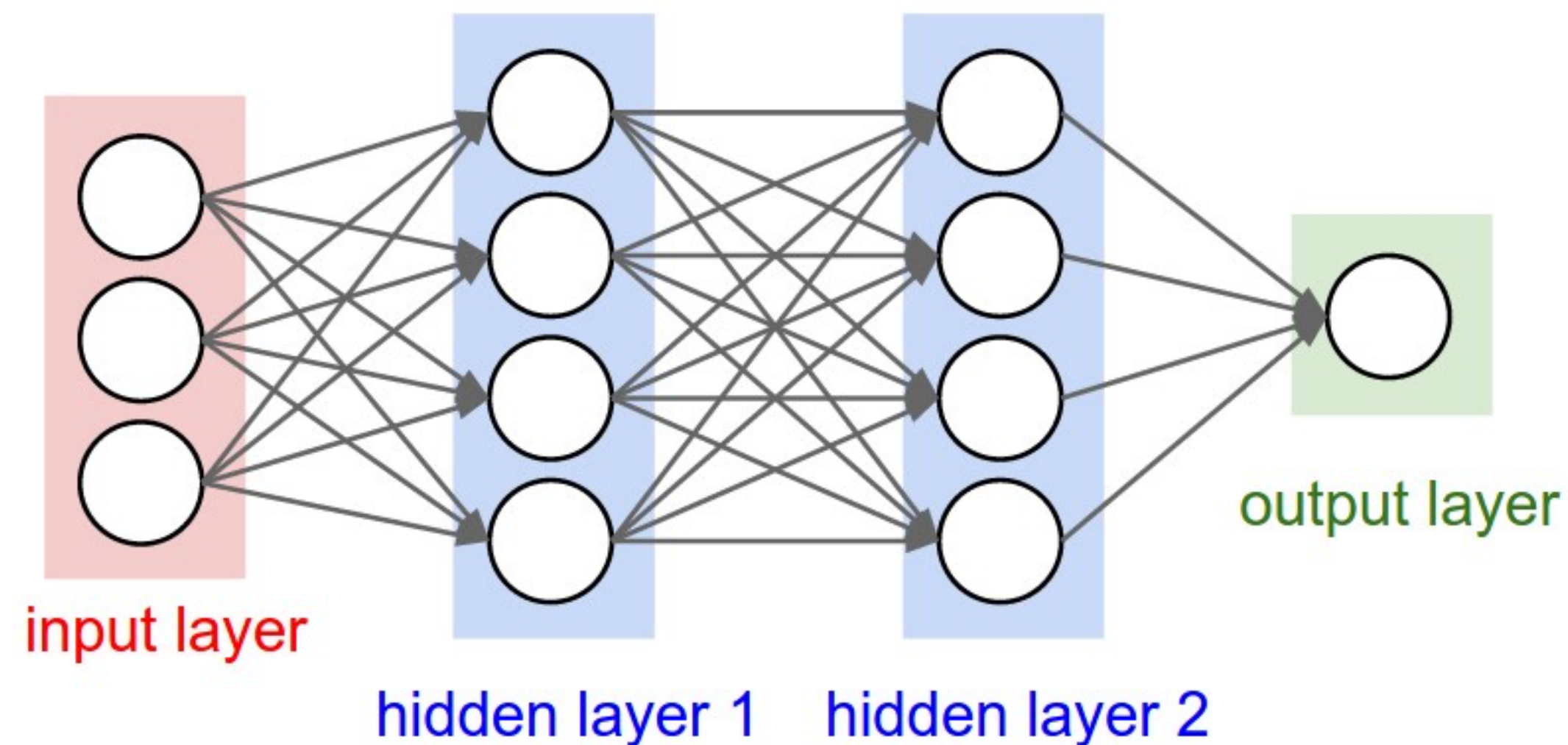
Convolution as Matrix-Vector Multiply

- A convolution can be written down as a matrix-vector multiply
- Matrix is very sparse; same filter weights are repeated in each row



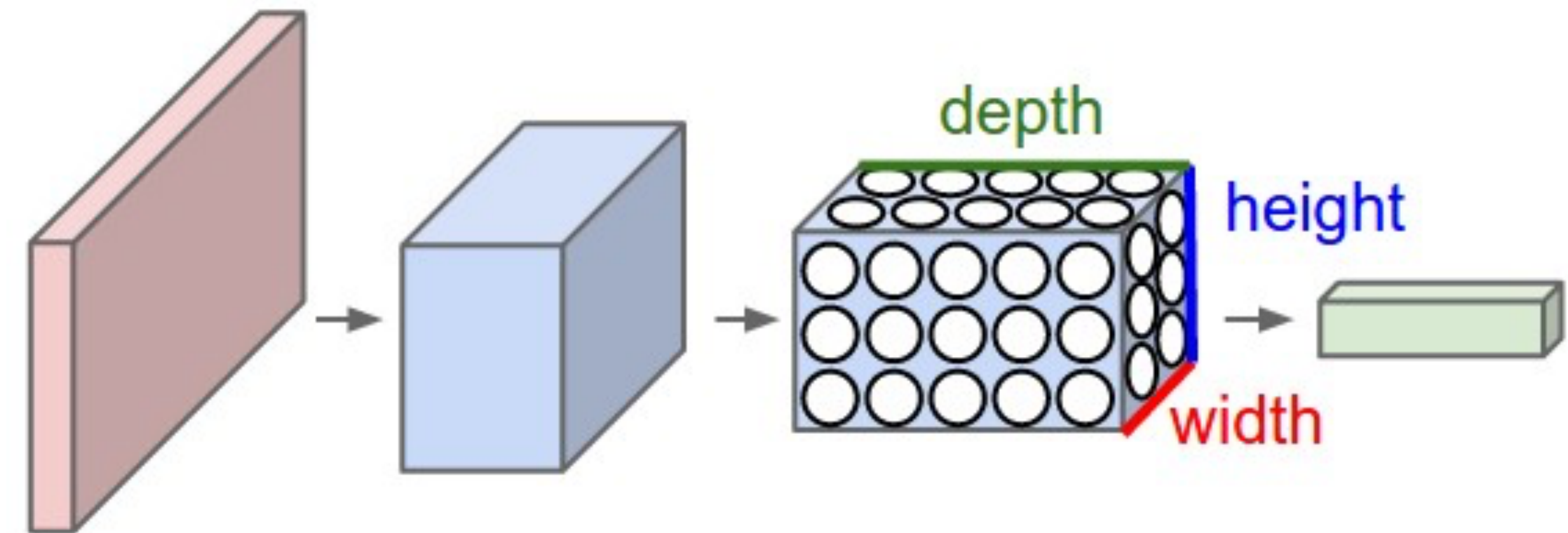
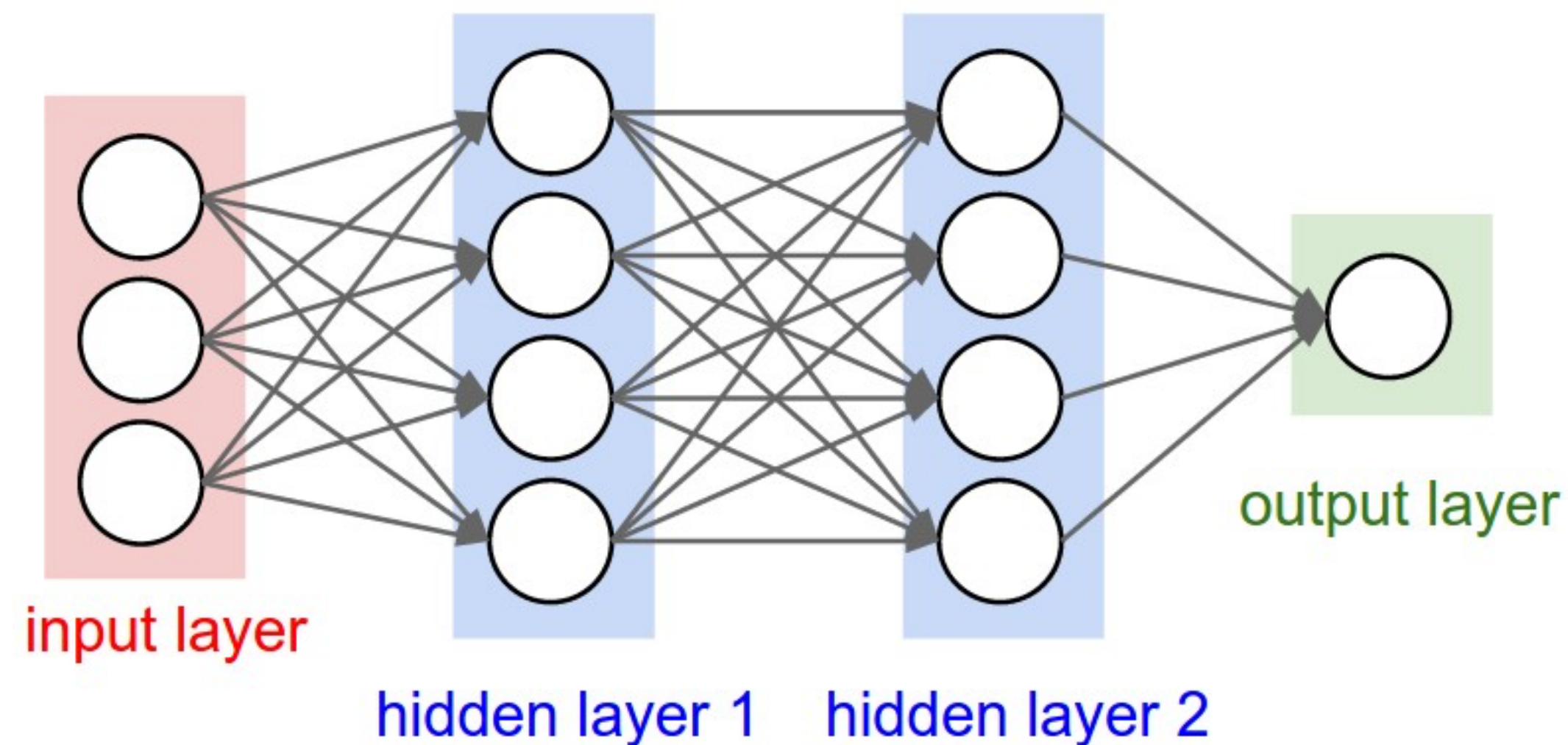
Convolutional Neural Network

- Convolutional Neural Network is equivalent to a traditional Neural Network
 - But, many neurons “share” weights (spatially-invariant property of convolution)
 - This greatly reduces the number of parameters in the network.



Convolution and pooling are an “infinitely strong” prior on the network architecture

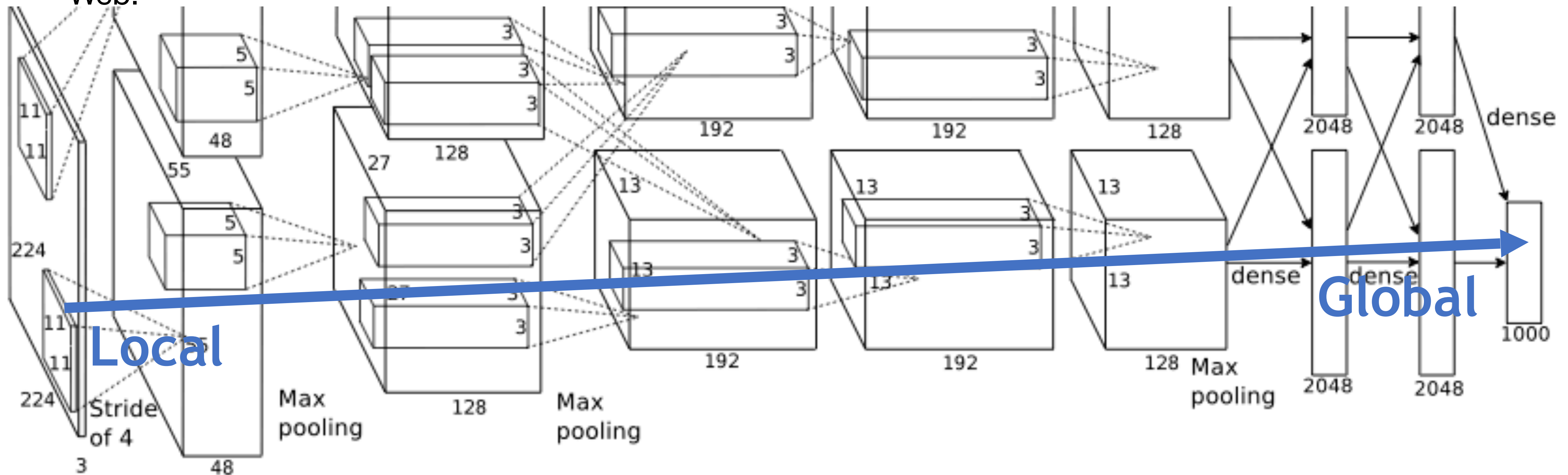
- “Prior:” beliefs about what models are reasonable, before seeing data
- This prior asserts that convolutional layers should only learn local interactions and should be equivariant to translation.



Case Study: AlexNet

- Winner of ImageNet 2012 -- launched the CNN revolution

Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E. Hinton. "ImageNet Classification with Deep Convolutional Neural Networks." *Advances in Neural Information Processing Systems* 25 (2012): 1–9. Web.



Filters learned by AlexNet

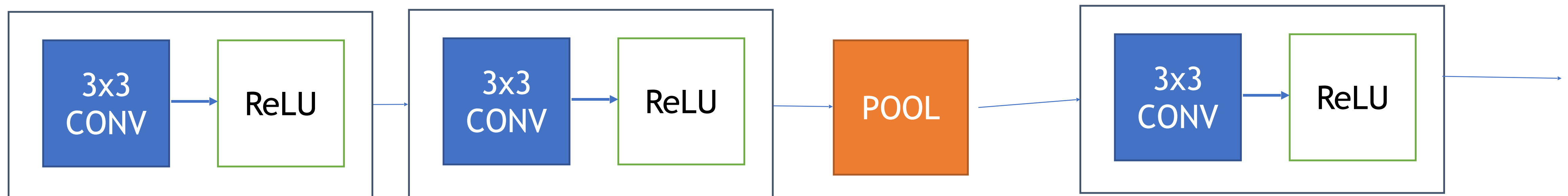
- Filters learned in first layer resemble Gabor filters (oriented band-pass)



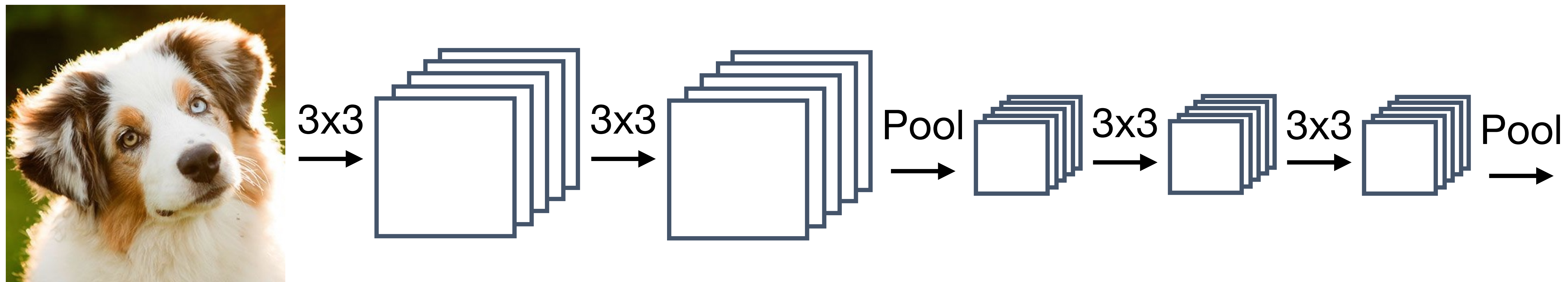
VGG Network

- Standard and straightforward architecture design
 - Only uses 3x3 convolutions with padding = 1 (“same” padding)
 - Each convolution followed by ReLU
 - 2x2, stride 2 max pooling after each group of convolutions
 - Double the number of channels after each max pooling (up to some max #)

Simonyan, Karen, and Andrew Zisserman. “Very Deep Convolutional Networks for Large-Scale Image Recognition.” *ICLR 2014*.



VGG Example



Input

512x512x3

Conv 1

512x512x32

Conv 2

512x512x32

Pool 1

256x256x32

Conv 3

256x256x64

Conv 4

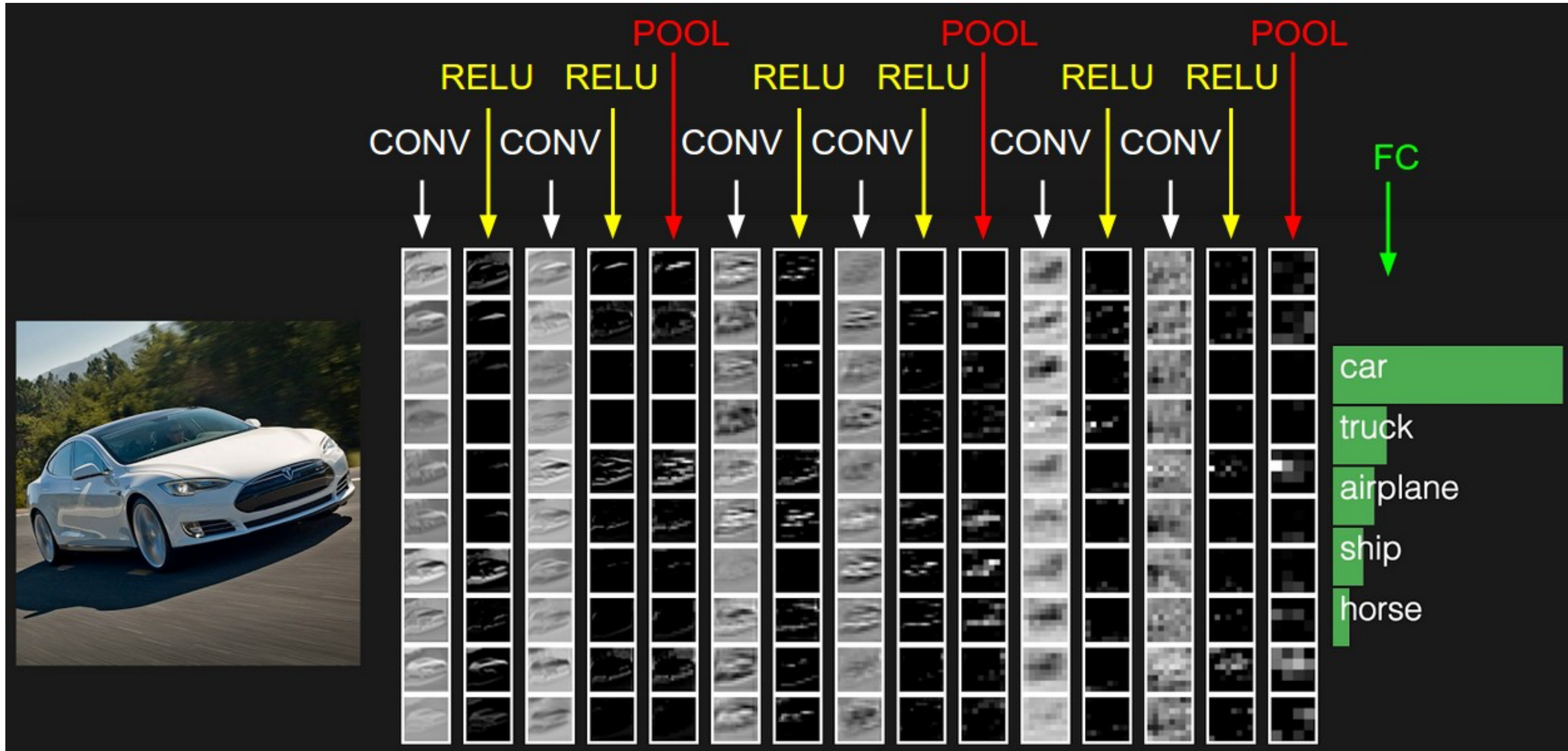
256x256x64

VGG Example

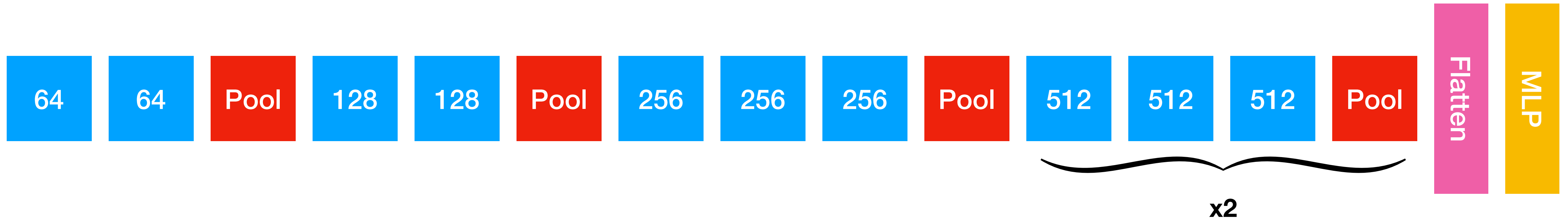
- Several blocks of conv-conv-pool
 - (Could have more or less convs)
- End of network:
 - Flatten to vector
 - Dense layers (fully connected)
 - Final output layer

Input	32x32x3
Conv2d 1a	32x32x32
Conv2d 1b	32x32x32
MaxPool2d	16x16x32
Conv2d 2a	16x16x64
Conv2d 2b	16x16x64
MaxPool2d	8x8x64
Conv2d 3a	8x8x128
Conv2d 3b	8x8x128
MaxPool2d	4x4x128
Flatten	2048
Linear 1	4096
Linear 2	4096
Linear 3	10

Case Study: "Tiny VGG Net"



VGG16 Network



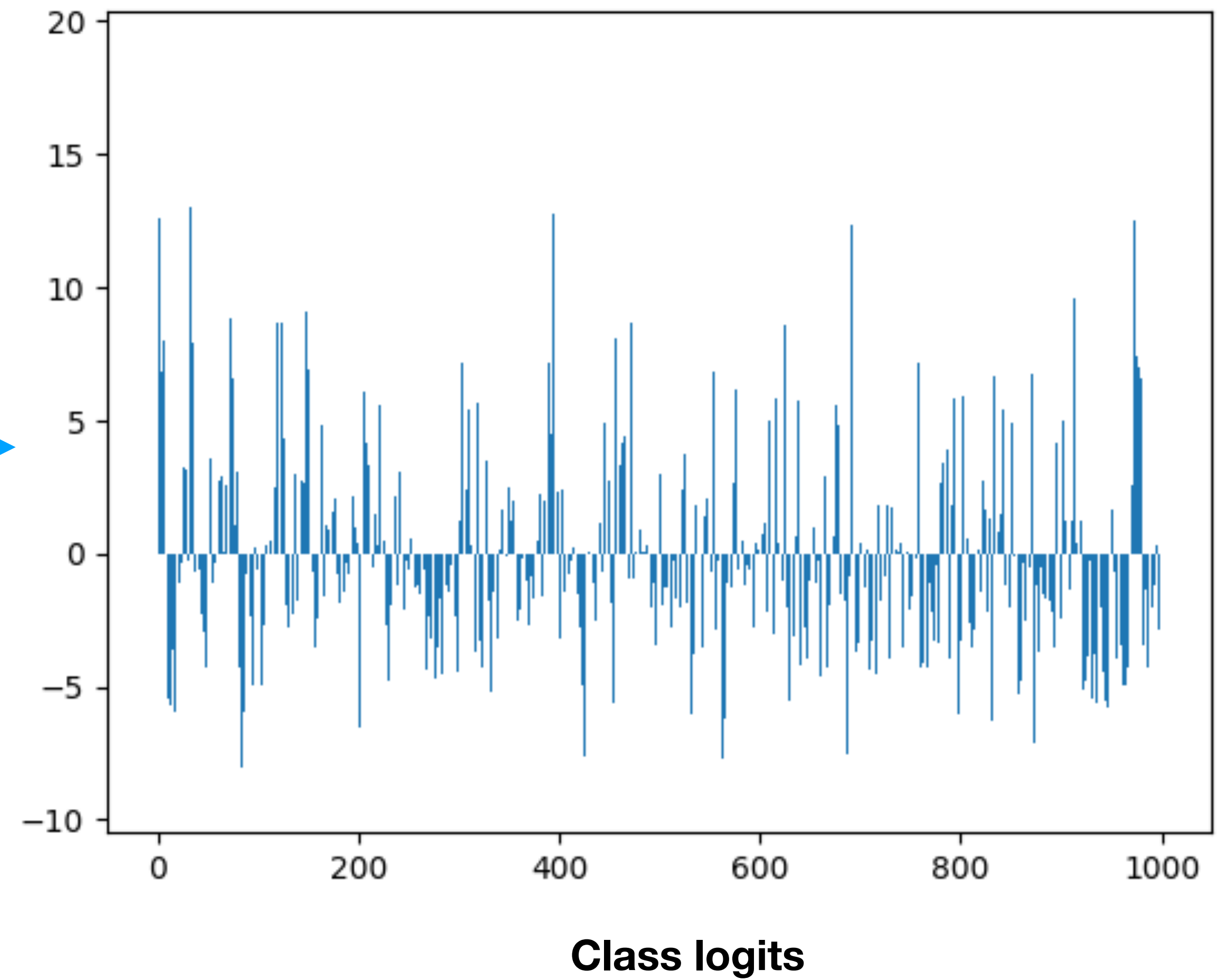
ImageNet classes

tench, *Tinca tinca*
goldfish, *Carassius auratus*
great white shark, white shark, man-eater, man-eating shark, *Carcharodon carcharias*
tiger shark, *Galeocerdo cuvieri*
hammerhead, hammerhead shark
electric ray, crampfish, numbfish, torpedo
stingray
cock
hen
ostrich, *Struthio camelus*
brambling, *Fringilla montifringilla*
goldfinch, *Carduelis carduelis*
house finch, linnet, *Carpodacus mexicanus*
junco, snowbird
indigo bunting, indigo finch, indigo bird, *Passerina cyanea*
robin, American robin, *Turdus migratorius*
bulbul
...

VGG16 Example



VGG16



VGG16 Example



VGG16

	classes	scores
0	tench, Tinca tinca	10.363527
1	goldfish, Carassius auratus	1.112662
2	great white shark, white shark, man-eater, man...	12.553168
3	tiger shark, Galeocerdo cuvieri	11.616830
4	hammerhead, hammerhead shark	6.807635
...	...	...
995	earthstar	0.351238
996	hen-of-the-woods, hen of the woods, Polyporus ...	0.089312
997	bolete	-2.860762
998	ear, spike, capitulum	-1.440554
999	toilet tissue, toilet paper, bathroom tissue	-3.497675

Class logits

VGG16 Example

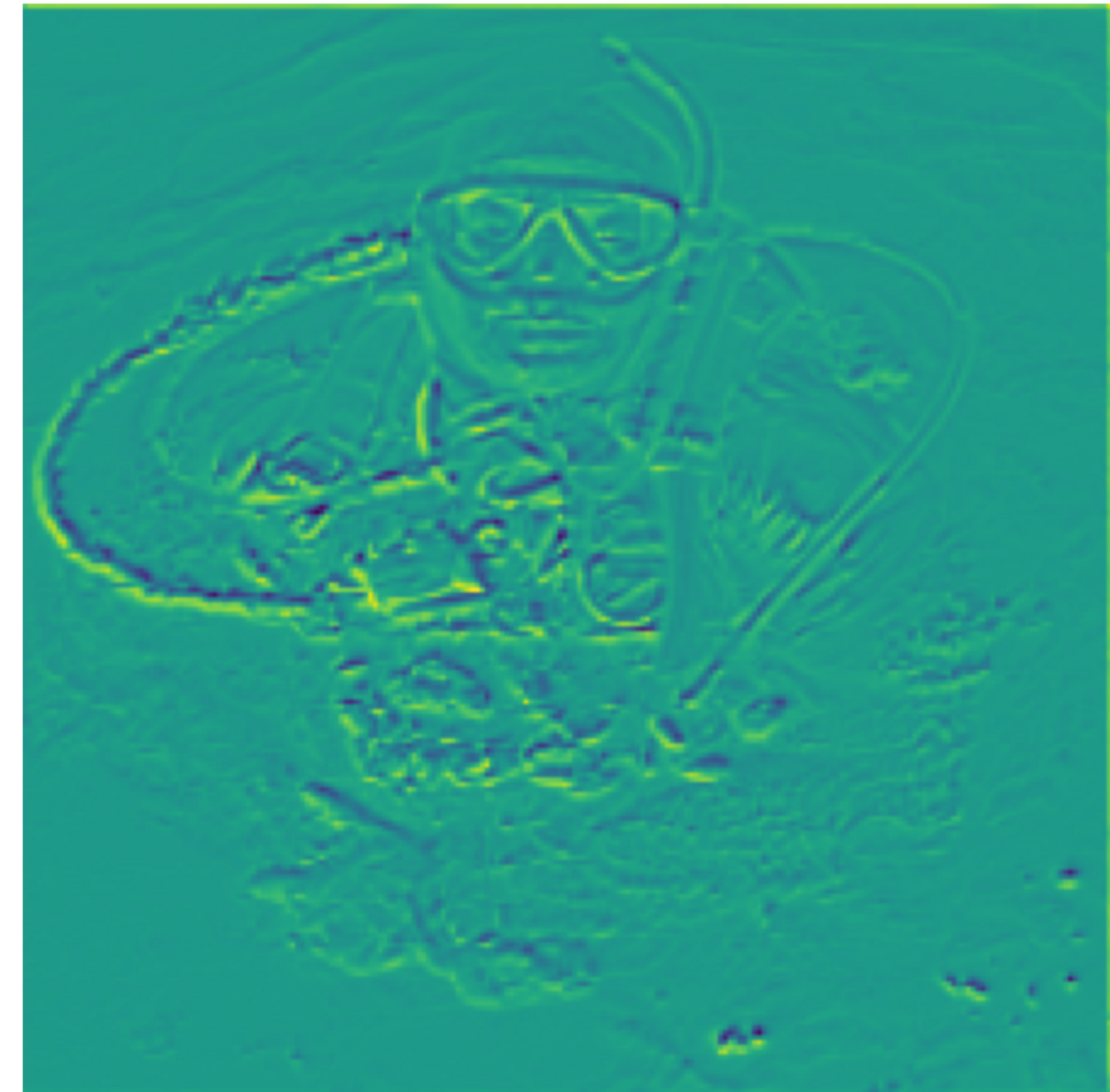
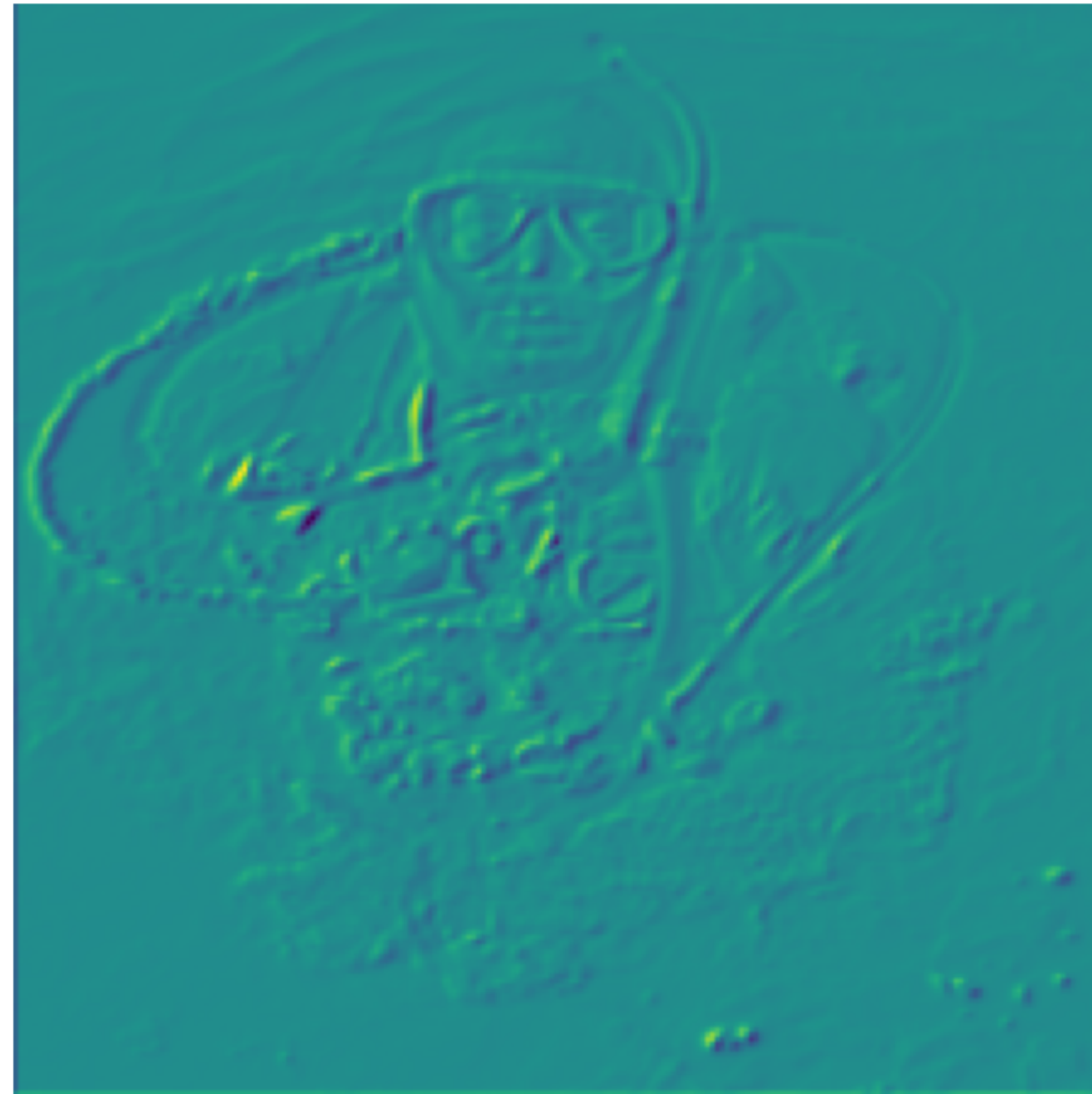


VGG16

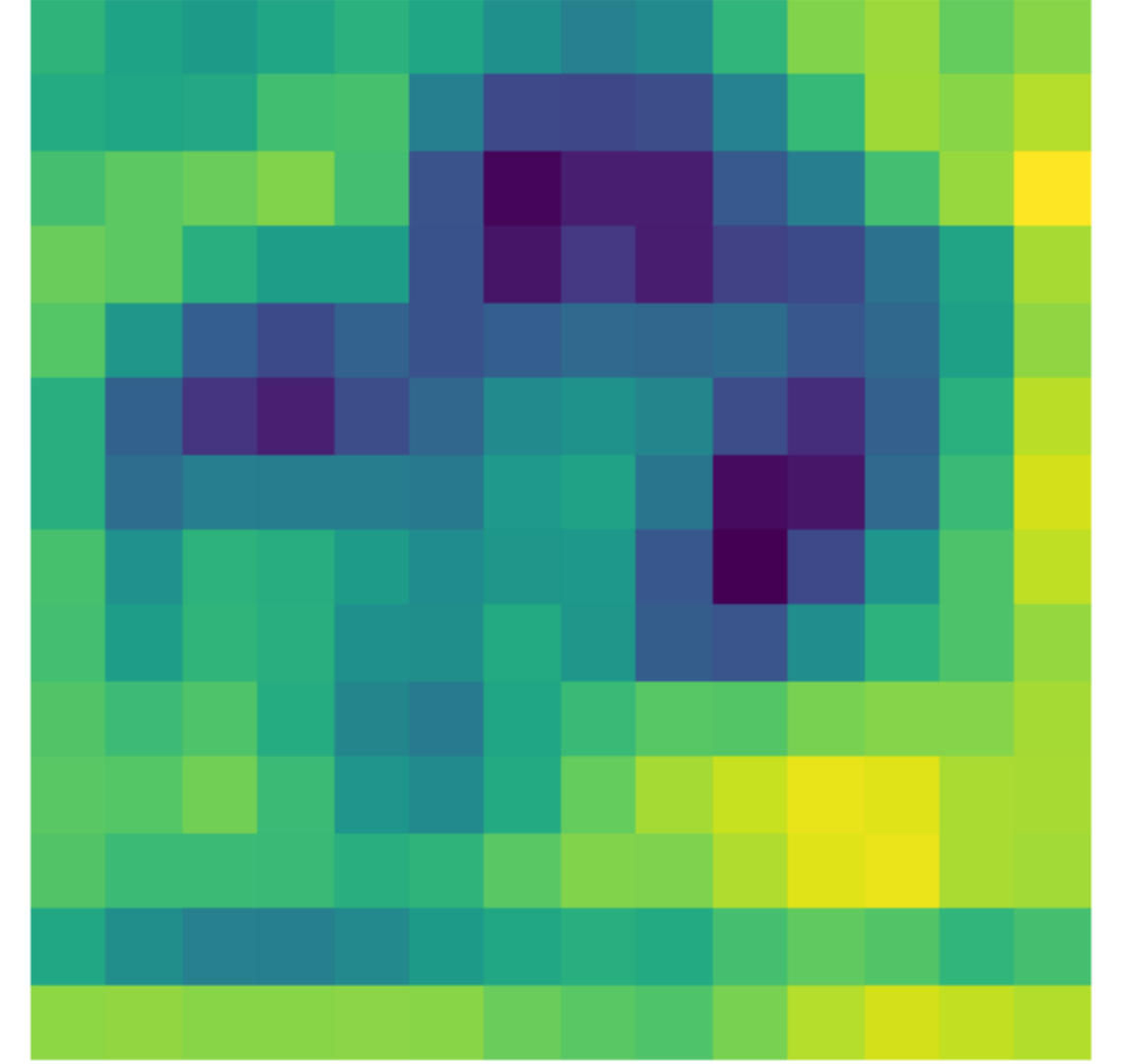
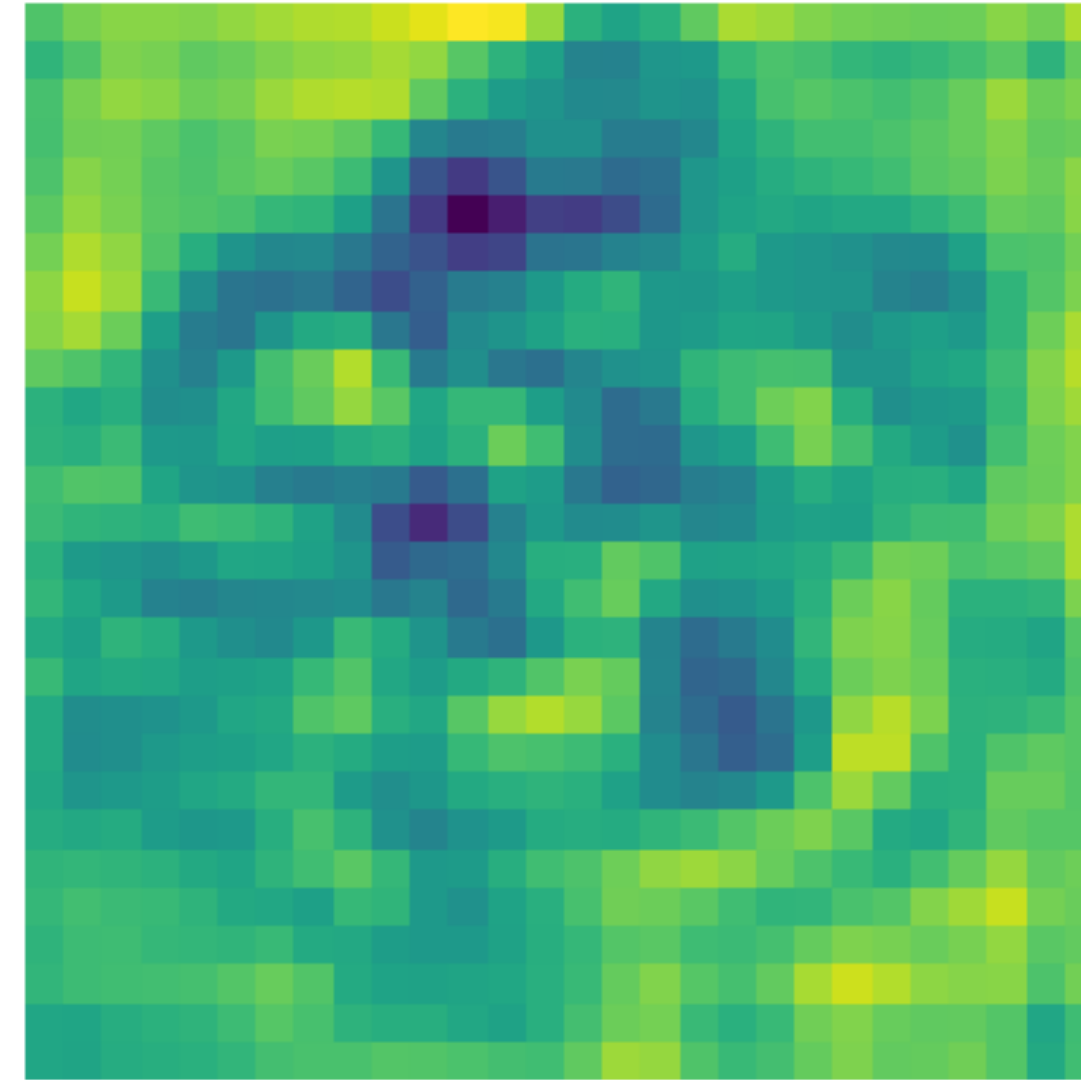
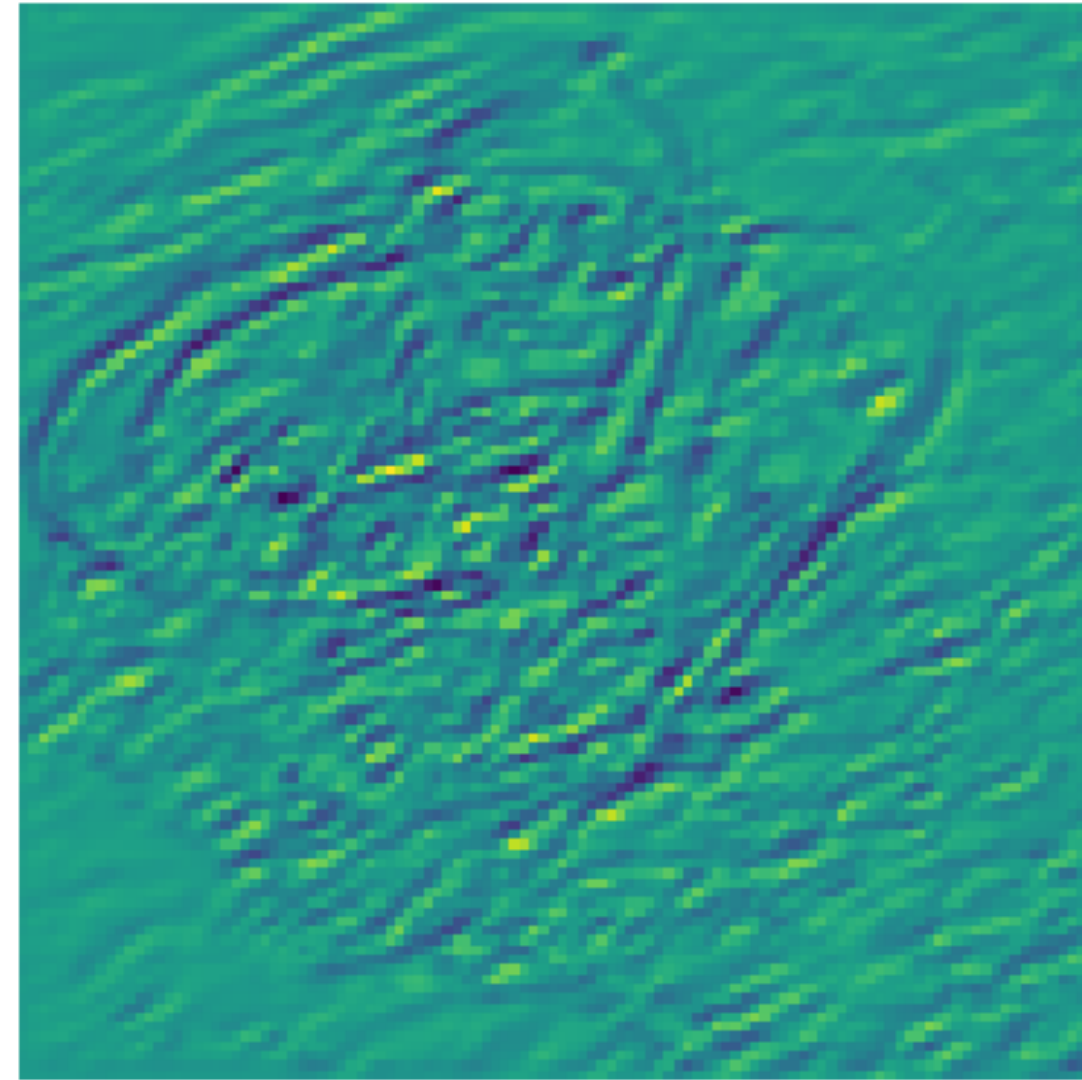
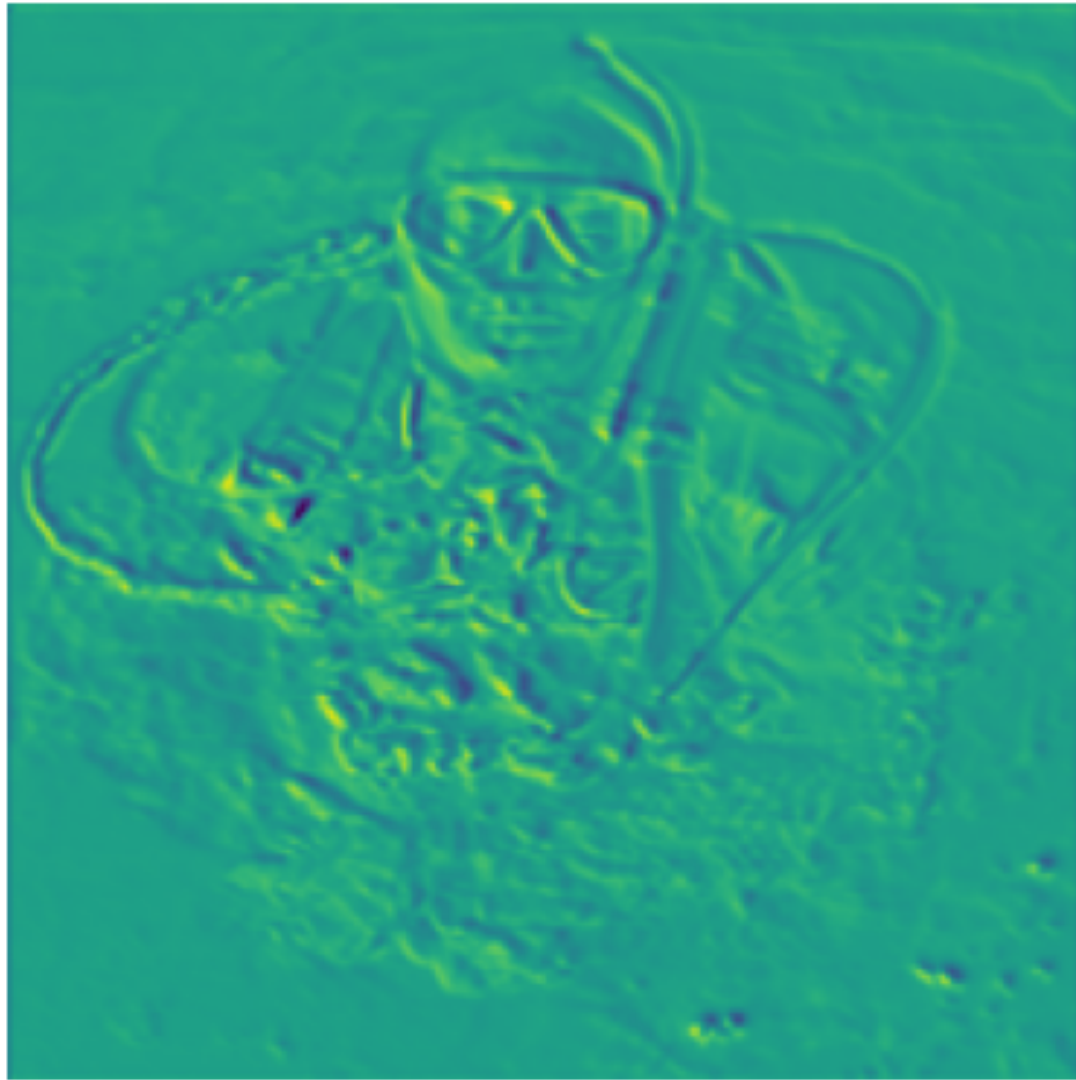
	classes	scores
0	tench, Tinca tinca	11.091852
973	coral reef	11.092249
6	stingray	11.113876
147	grey whale, gray whale, devilfish, Eschrichtiu...	11.236611
540	drilling platform, offshore rig	11.601207
394	sturgeon	11.921114
329	sea cucumber, holothurian	12.001931
691	oxygen mask	13.048210
801	snorkel	14.833947
983	scuba diver	19.271877

Sorted class logits

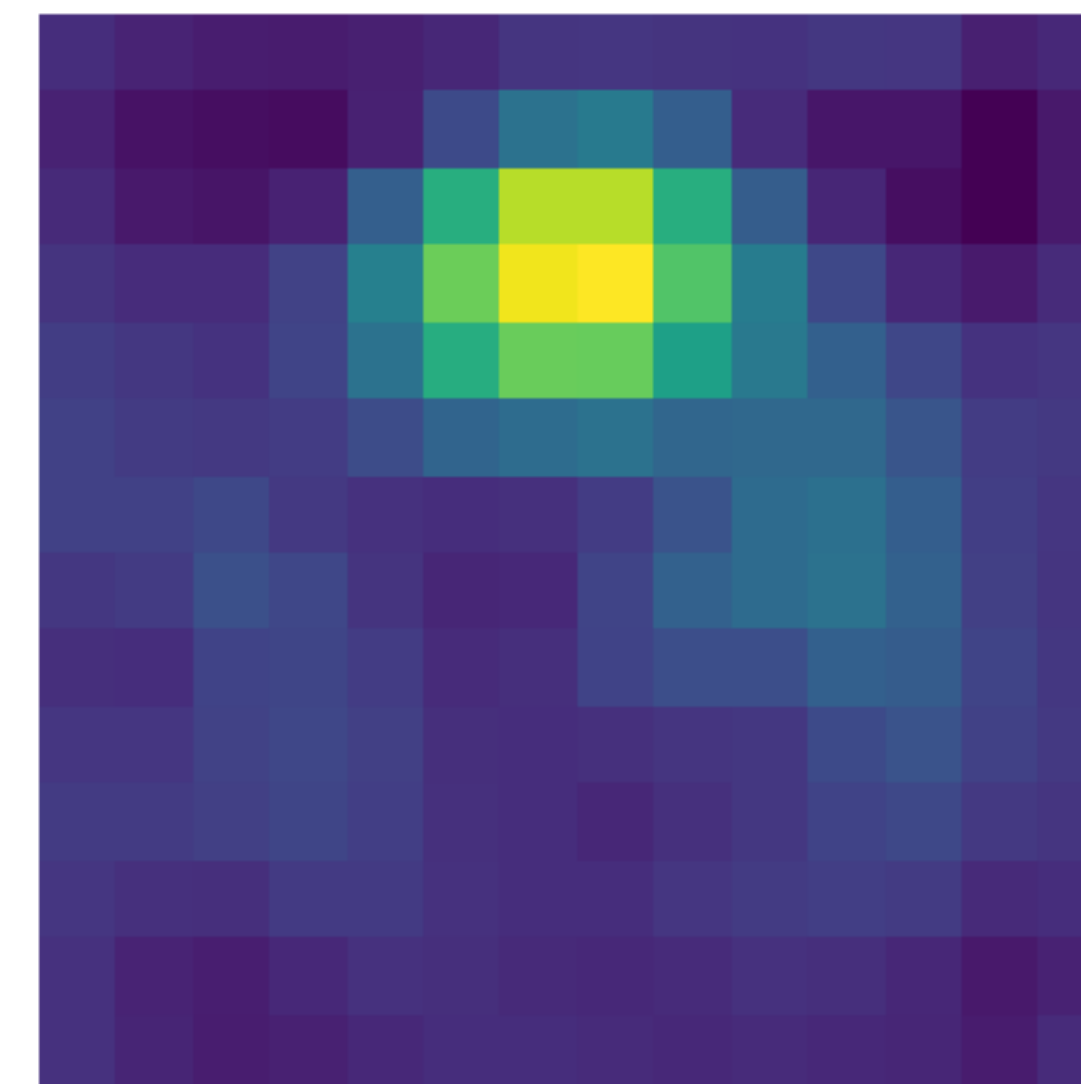
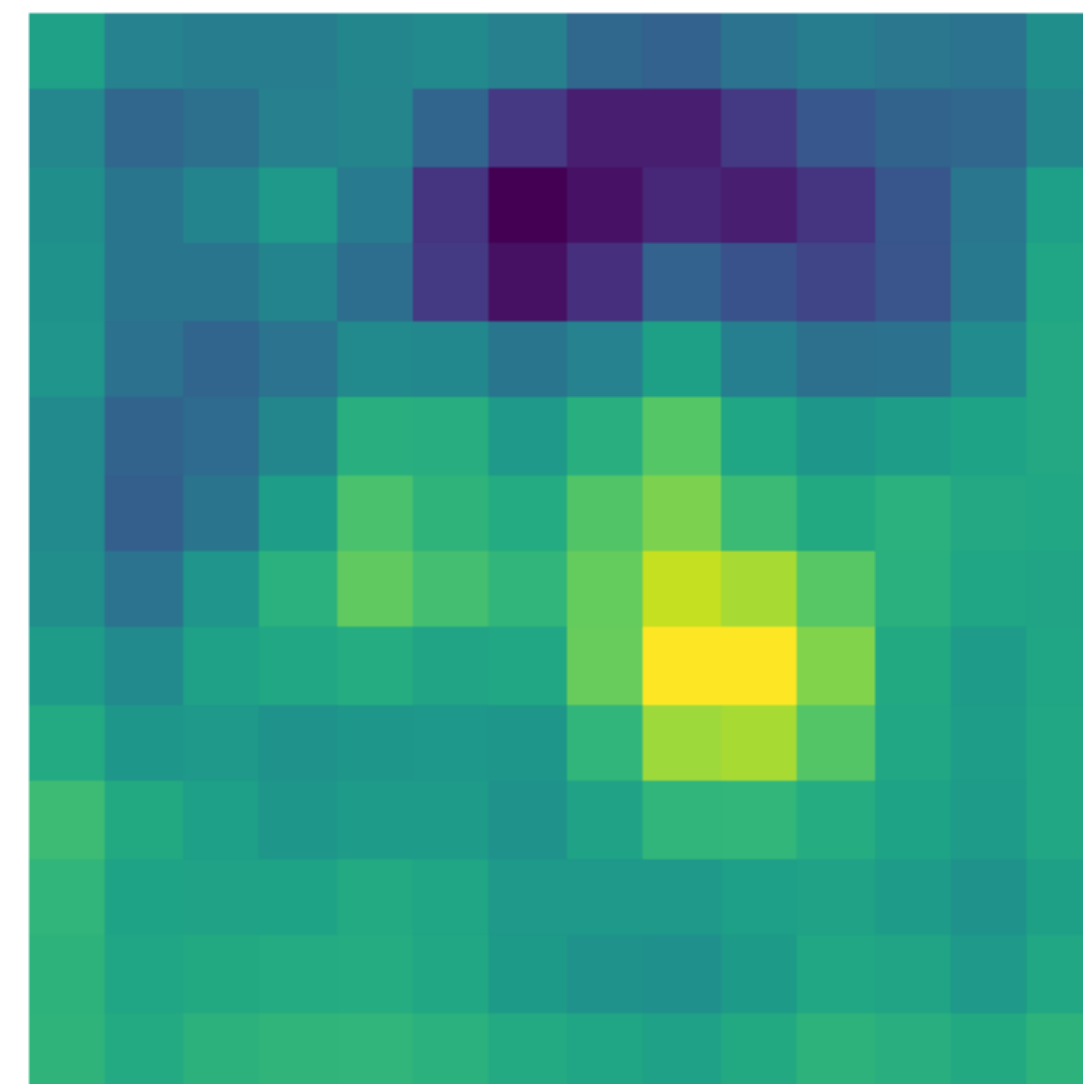
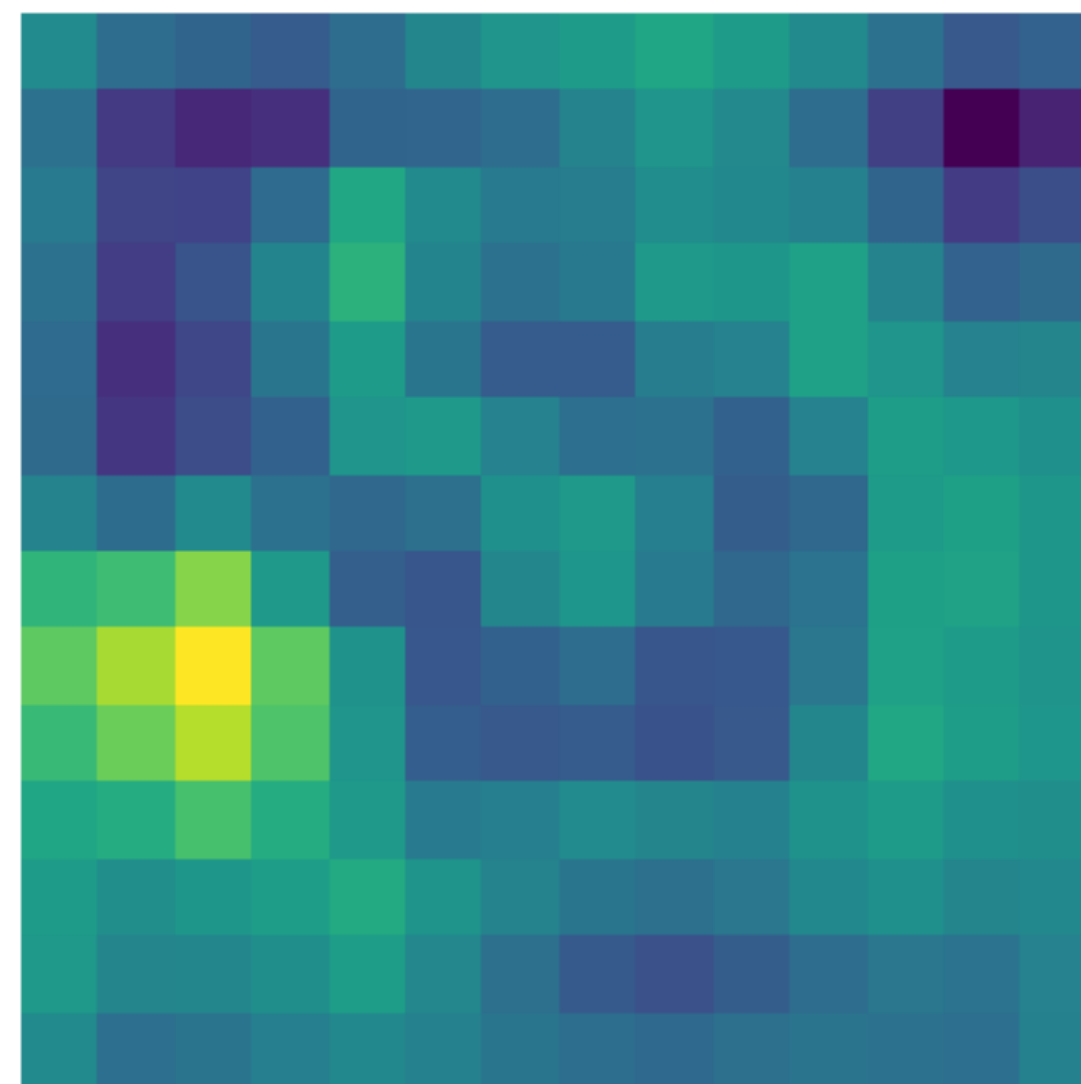
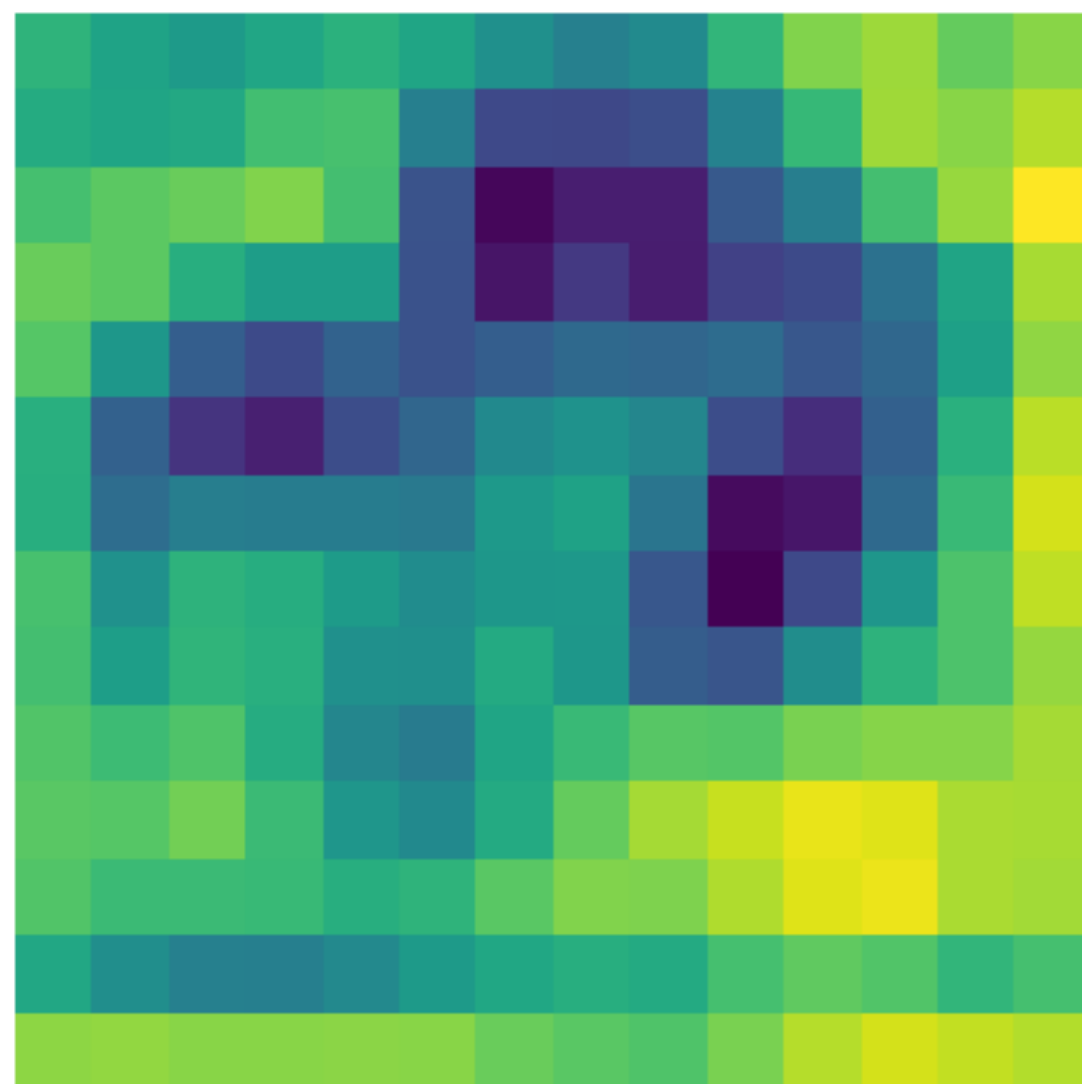
First layer filter responses



Later layers

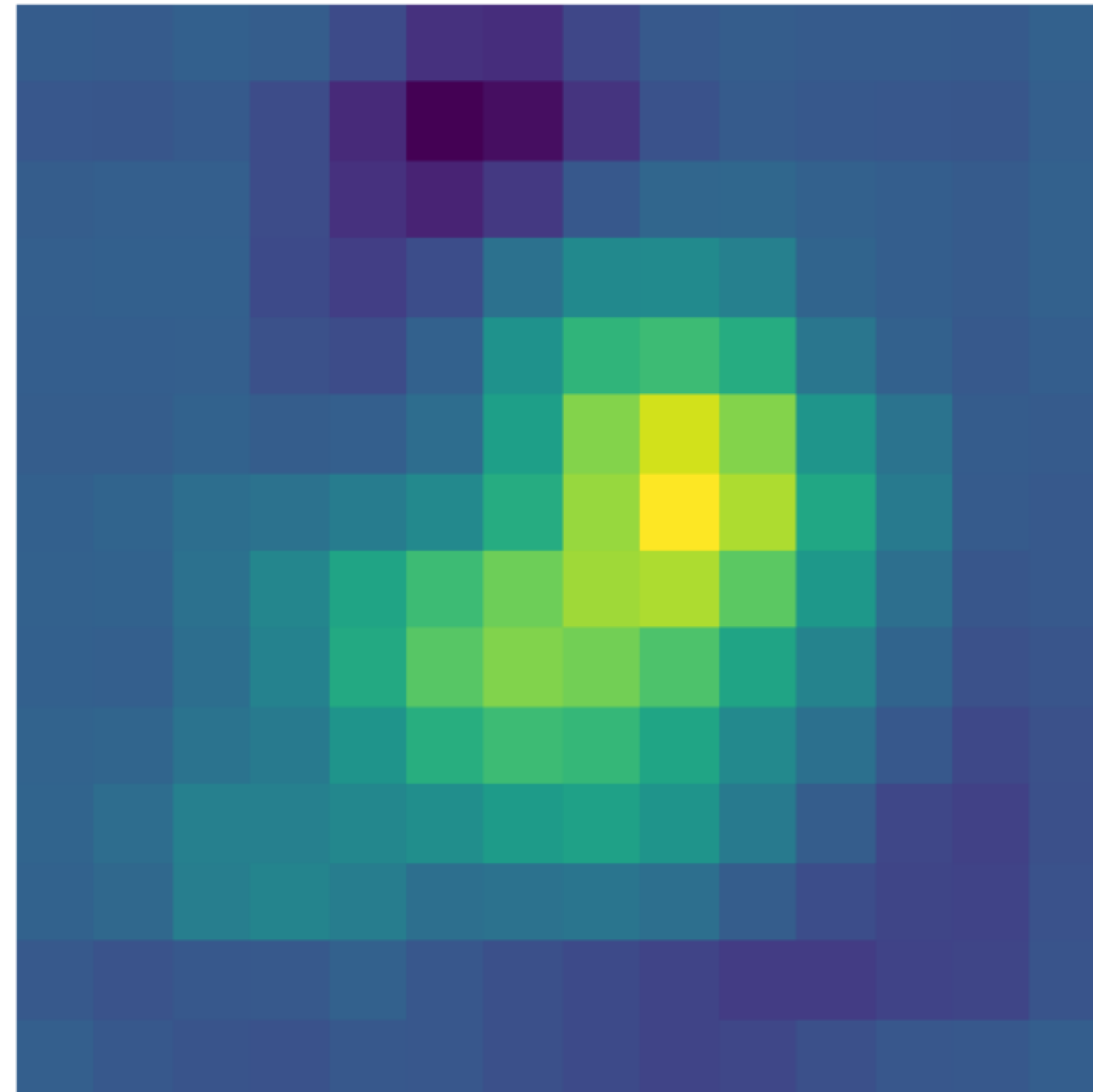


Feature maps in last layer



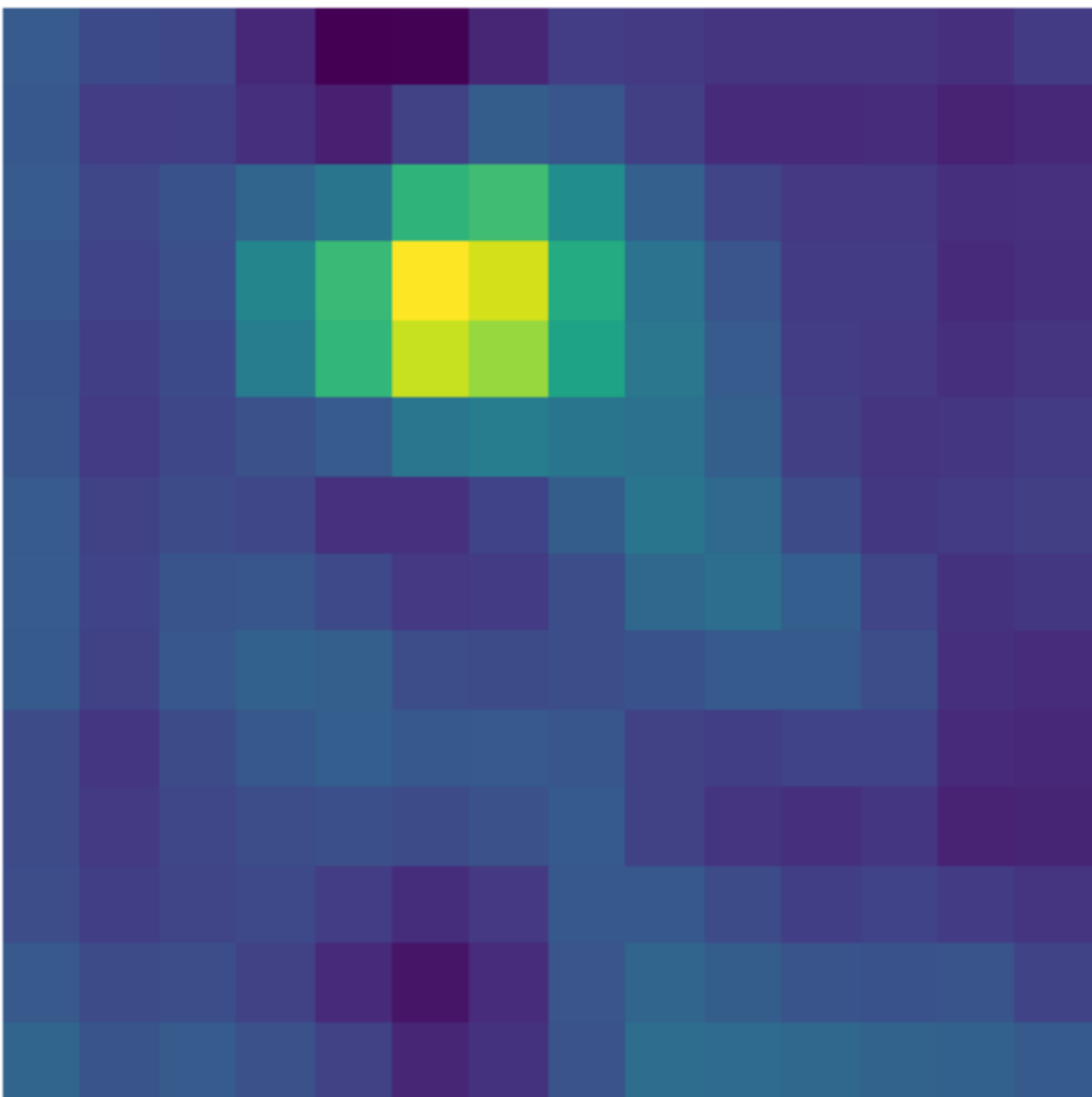
**Feature map with
largest activation**

More examples



	classes	scores
787	shield, buckler	12.967152
676	muzzle	15.044091
558	flute, transverse flute	16.191071
420	banjo	16.463203
432	bassoon	17.684416
683	oboe, hautboy, hautbois	21.321440
776	sax, saxophone	22.909592
875	trombone	25.383024
513	cornet, horn, trumpet, trump	27.706257
566	French horn, horn	36.337456

More examples



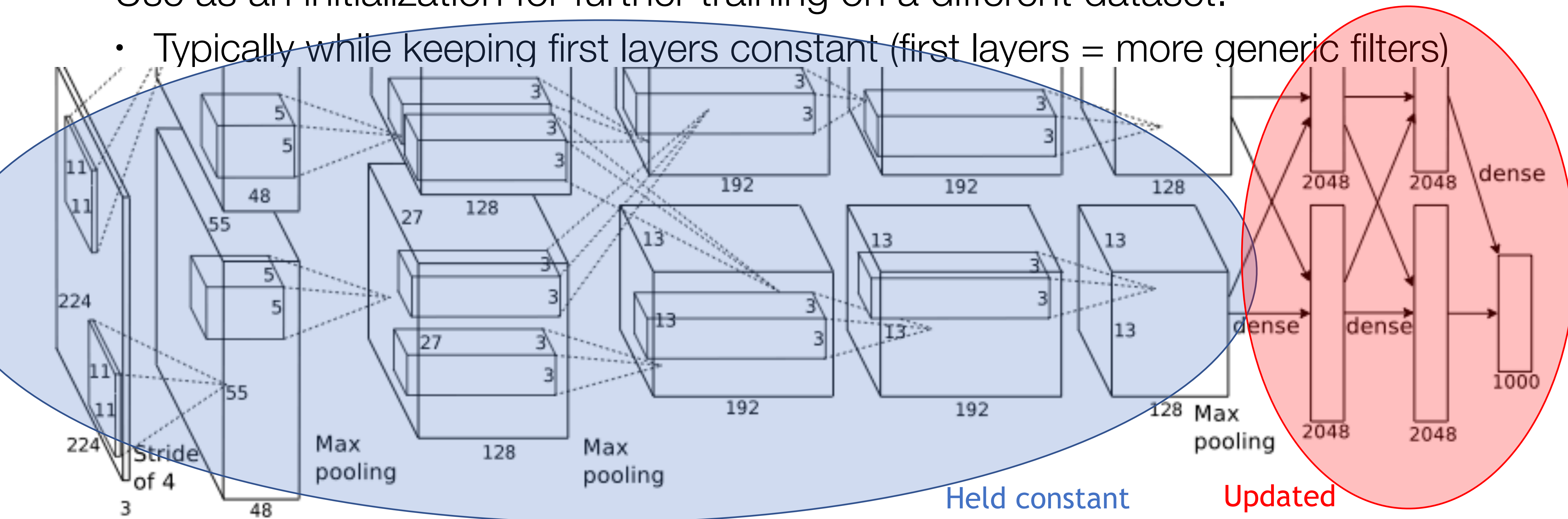
	classes	scores
287	lynx, catamount	12.192641
22	bald eagle, American eagle, Haliaeetus leucoce...	12.591387
93	hornbill	12.982509
282	tiger cat	13.249150
377	marmoset	13.801717
86	partridge	15.083697
82	ruffed grouse, partridge, Bonasa umbellus	15.725797
21	kite	15.810165
83	prairie chicken, prairie grouse, prairie fowl	16.458763
24	great grey owl, great gray owl, Strix nebulosa	34.610683

Choosing a CNN Architecture

- Many considerations:
 - What size filters?
 - How many layers?
 - Where to put pooling layers?
 - How many hidden layers and how big?
- Advice from Andrej Karpathy: “Don’t be a hero”
 - Check which network is currently doing best on ImageNet
 - Adapt that network to your problem (fine-tune for your problem)

Fine-tuning

- Take an existing network trained on one dataset (e.g. AlexNet)
- Use as an initialization for further training on a different dataset.
 - Typically while keeping first layers constant (first layers = more generic filters)



Getting “more data”: Data Augmentation

- Randomly transform your training images:
 - Flip, rotate, scale, translate, brightness, contrast, ...

